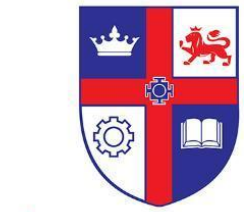
LONDON METROPOLITAN UNIVERSITY



Islington college
(इस्लिङ्टन कलेज)

CS4001NI Programming

60% Individual Coursework

AY: 2024-25 Spring

Credit: 30

Student Name: Alina B.K.

London Met ID: 24058871

College ID: NP01CP4S250039A

Assignment Due Date: Friday, August 29, 2025

Assignment Submission Date: Friday, August 29, 2025

Word Count: 7,956

I confirm that I understand my coursework needs to be submitted online via MySecondTeacher under the relevant module page before the deadline in order for my assignment to be accepted and marked. I am fully aware that late submissions will be treated as non-submission and a marks of zero will be awarded.

Table of Contents

Table of Figures

List of Table

5% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Match Groups

- **23** Not Cited or Quoted 5%
  Matches with neither in-text citation nor quotation marks
- **0** Missing Quotations 0%
  Matches that are still very similar to source material
- **0** Missing Citation 0%
  Matches that have quotation marks, but no in-text citation
- **0** Cited and Quoted 0%
  Matches with in-text citation present, but no quotation marks

Top Sources

1% Internet sources
0% Publications
5% Submitted works (Student Papers)

Integrity Flags

0 Integrity Flags for Review

Our system's algorithms look deeply at a document for any inconsistencies that would set it apart from a normal submission. If we notice something strange, we flag it for you to review.

A Flag is not necessarily an indicator of a problem. However, we'd recommend you focus your attention there for further review.

Introduction

Course work assignment is to create and implement a complete Art Gallery Management System in Java demonstrating good usage of object-oriented programming (OOP) concepts. Art galleries of today's web world require good systems of customer management, purchase registration, discount, and proper bills, and this assignment simulates a sample actual system of this kind. The program invokes an abstract base class (ArtGalleryVisitor) and its derived classes (StandardVisitor and EliteVisitor) for illustrating principles of abstraction, inheritance, polymorphism, and encapsulation. The system uses these classes to store visitor data like visits, purchase history, cancellation history, and pending offers or reward points. Interactive Graphical User Interface (GUI) has been obtained with Java Swing and AWT widgets such that the interface is designed self-explaining and visitor details can be input, printed, and processed in auto-descriptive mode. GUI makes frequent use of text fields, combo boxes, radio buttons, and tables and hence employs considerable amounts of Swing functionality. Apart from this, the system also makes use of file handling functionality to save visitor details and is never deleted even when the application is closed. Exception handling using try–catch blocks renders the system fault-tolerant by not allowing it to crash upon invalid or incomplete detail entries. Comprehensive testing of program correctness and stability is conducted, and purchase, cancellation, billing, and reward points calculation use cases are being tested. Overall, the project will enhance problem-solving skills and coding skills and enhance a general learning process of software development.

Amis and Objectives

This assignment strictly follows the Academic Misconduct and Integrity Statement (AMIS). This work is original, composed without unauthorized assistance or plagiarism of any sort from any outside source. Any references cited, should they be used, are properly referenced, whilst programming execution, diagrams, and testing have been done personally to maintain academic honesty and integrity.

- To design an object-oriented system in Java to simulate a visitor management system for an art gallery.
- To apply OOP concepts of abstraction, inheritance, and polymorphism by developing abstract and concrete visitor classes.
- To simulate and implement a GUI through Java Swing and AWT components.
- To apply the appropriate Swing components like text fields, combo boxes, and radio buttons to accept user data.
- To manage visitor data dynamically using an ArrayList that holds StandardVisitor and EliteVisitor objects.
- To use try-catch blocks to avoid unwanted or invalid user input and render the program more robust.

Technology Used

1. **Java**

   Java is an object-oriented, class-based, high-level programming language based on the idea of "Write Once, Run Anywhere" (WORA). Java implies that compiled code can be run on any device or computer system with the Java Virtual Machine (JVM) installed, without platform recompilation.

Figure 1: Java Logo

2. **BlueJ**

BlueJ is an open source, introductory Integrated Development Environment (IDE) for Java, which makes it simple to learn object-oriented programming with its interactive interface, visual class diagrams, and friendly features like an in-built editor, compiler, and debugger.

Figure 2: BlueJ

3. **MS Word**

Microsoft Word is a word processing program created by Microsoft. It is a component of the Microsoft Office software and is used to type, edit, format, and print documents like letters, reports, resumes, etc. It provides functionalities like spelling check, grammar check, formatting, and adding images, tables, and charts.

Figure 3: MS Word Logo

4. **Notepad**

Notepad is a text editor that is included with Windows. It's used for writing, editing, and viewing plain text documents like notes, scripts, or basic web pages. It's lightweight, fast-loading, and includes basic features like saving, opening, and basic editing functionality.

Figure 4: Notepad Logo

5. **Draw.io**

Figure 5: Draw.io Logo

Draw.io is an open-source, web-based, and free diagramming tool for creating professional-level flowcharts, mind maps, network graphs, and other kinds of visualization. It has a convenient drag-and-drop interface, a large library of shapes and templates, real-time collaboration capabilities, and seamless integration with major cloud storage systems. You can install it as a web application, desktop software, or browser extension, and it boasts widespread usage in software development, business, and engineering.

GUI Wireframe

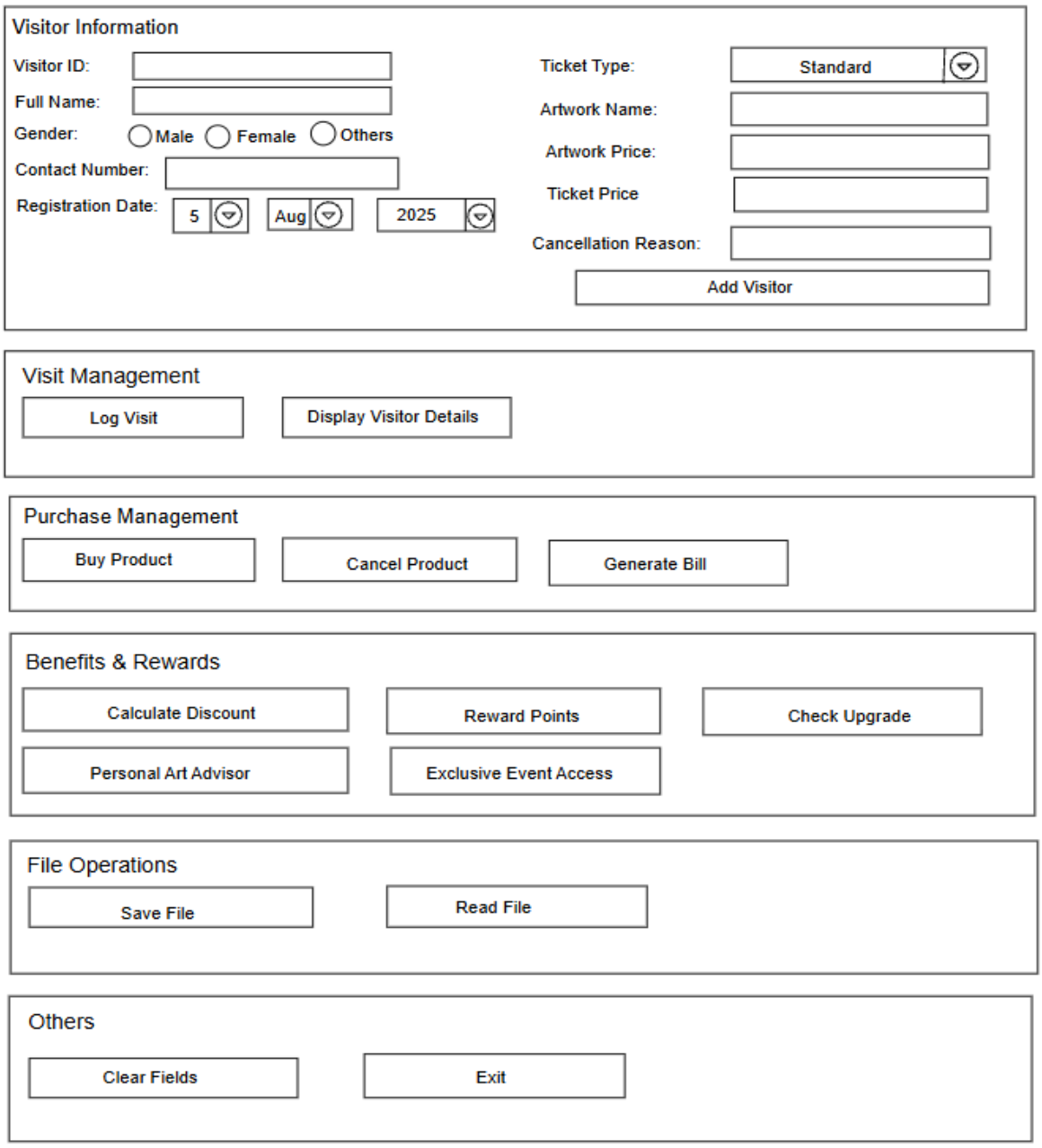


Figure 6: GUI Wireframe

Class Design and Implementation

ArtGalleryVisitor Class (Abstract Class)

The ArtGalleryVisitor class is a virtual parent class that stores common data of all the visitors such as ID, name, gender, contact, registration date, type of ticket, and ticket price. It also stores visit count, purchase history, cancellations, discounts, reward points, and activity status. The class contains a constructor to initialize the values and getter methods for all the attributes. It defines abstract methods to buy, cancel, calculate discount and reward, and generate bills, which are implemented by its subclasses. It has a logVisit() method and a display () method to print visitor information.

Figure 7: Class Diagram of ArtGalleryVisitor

EliteVisitor Class

The EliteVisitor class is an extension of ArtGalleryVisitor and represents high-end visitors with extra privileges. It has assignedPersonalArtAdvisor and exclusiveEventAccess features, which activate after sufficient reward points are earned. Elite patrons enjoy a higher discount of 40% and earn 10 reward points for each rupee spent. The class overrules purchase, discount, rewards, cancellations with only a 5% fee, and bill generation. The display() method prints both typical visitor details and elite-member exclusive permissions, and a private terminateVisitor() restarts the account upon violation of cancellation limits.

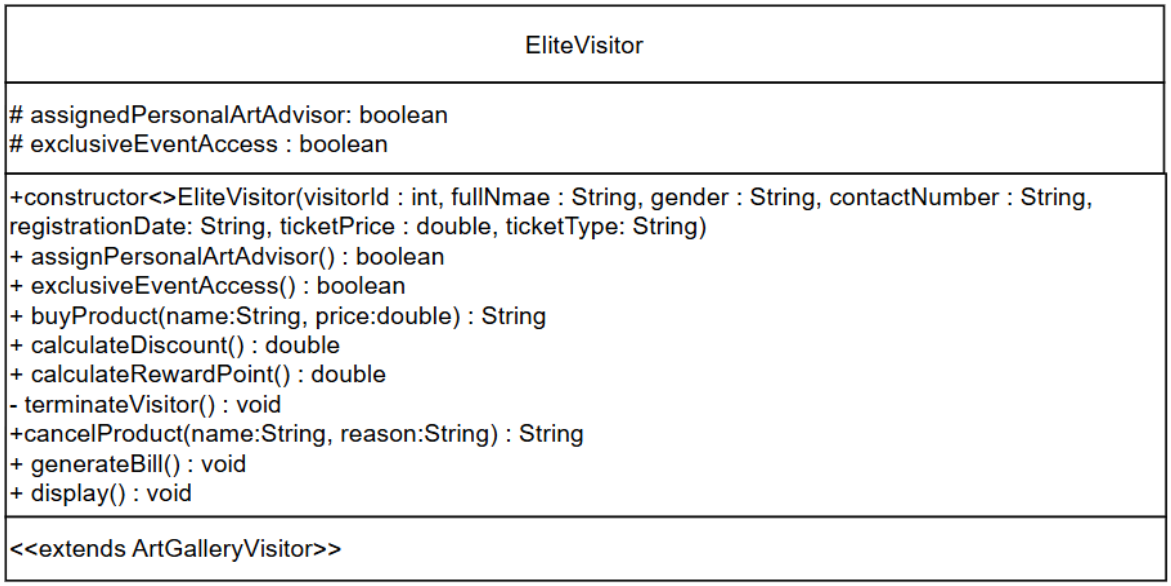


Figure 8: Class Diagram of EliteVisitor

StandardVisitor Class

The StandardVisitor class is an extension of ArtGalleryVisitor and represents normal visitors who start with a 10% discount on art pieces. It introduces fields such as discountPercent, visitLimit, and isEligibleForDiscountUpgrade. The class raises the discount to 15% on fifth visits and awards 5 reward points for each rupee spent. It overrides the abstract methods to process purchases, cancellations at a 10% fee, and bill generation. The display() method prints parent and subclass messages, and a private terminateVisitor() inhibits further use after several cancellations.

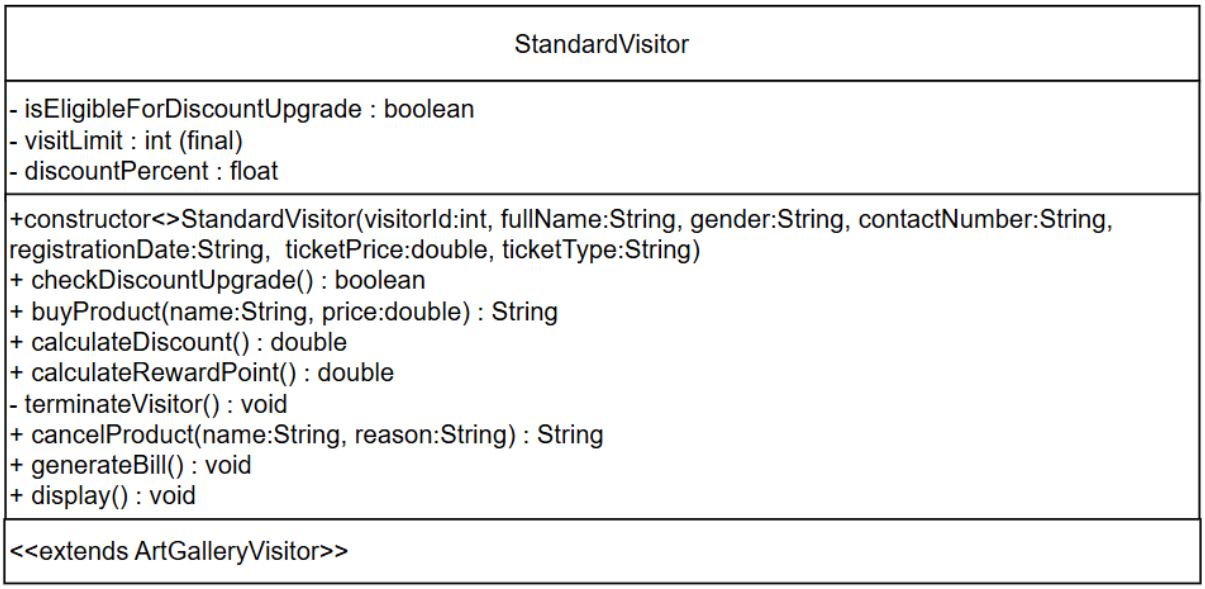


Figure 9: Class Diagram of StandardVisitor

GUI Overview

The ArtGalleryGUI class implements a graphical user interface using Java Swing for managing visitors of either type. It provides text fields, combo boxes, radio buttons, and action buttons for performing operations like adding visitors, logging visits, buying or cancelling products, checking upgrades, and generating bills. Visitor details are stored in an ArrayList and the application avoids duplicate visitor IDs by using exception handling. File operations to write and read visitor details and display bills or records in dialogs and tables are also included in the GUI. It integrates the backend logic with a user-friendly interface to handle the art gallery system.

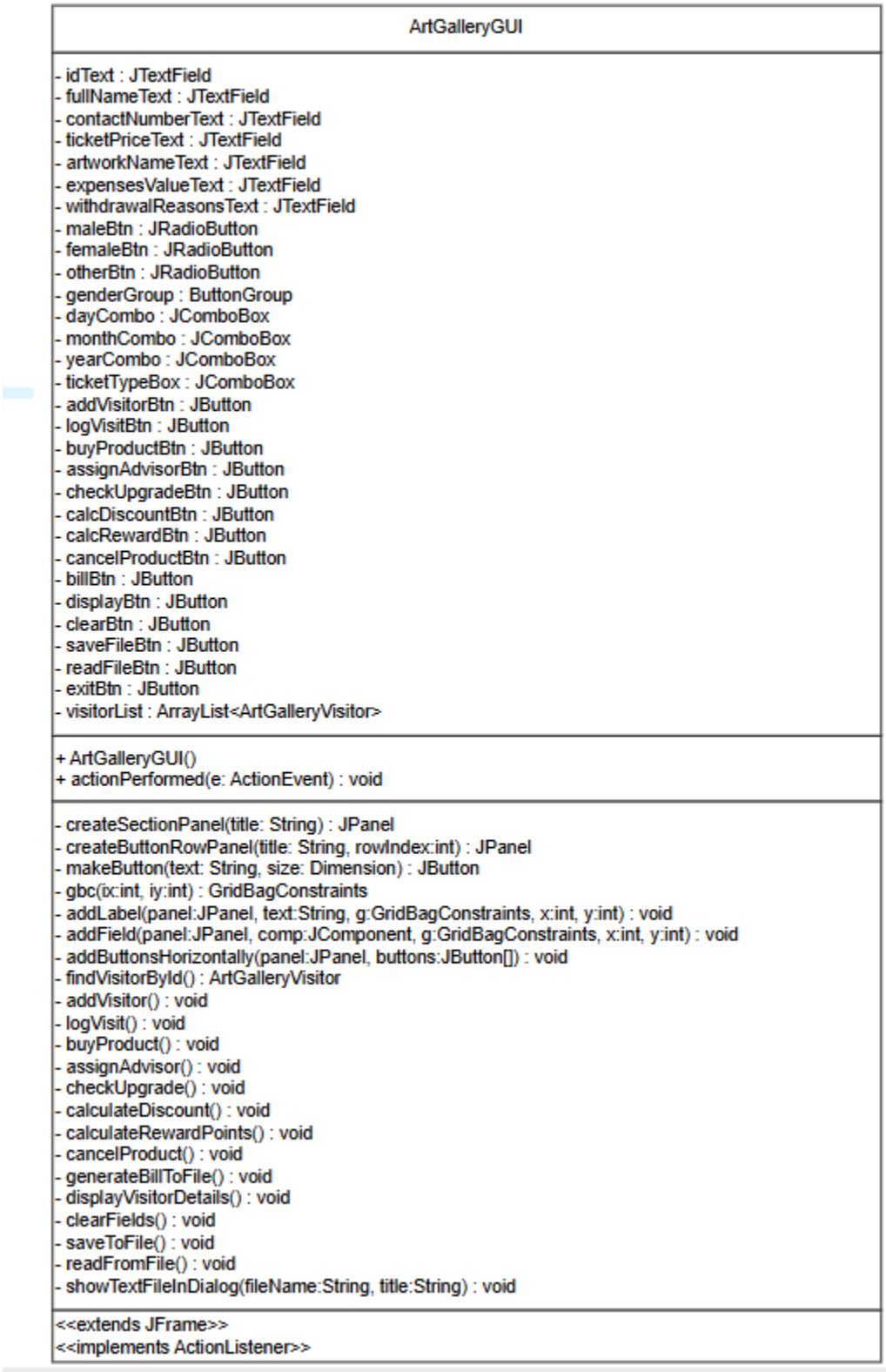


Figure 10: Class Diagram of ArtGalleryGUI

UML Class Diagram Representation

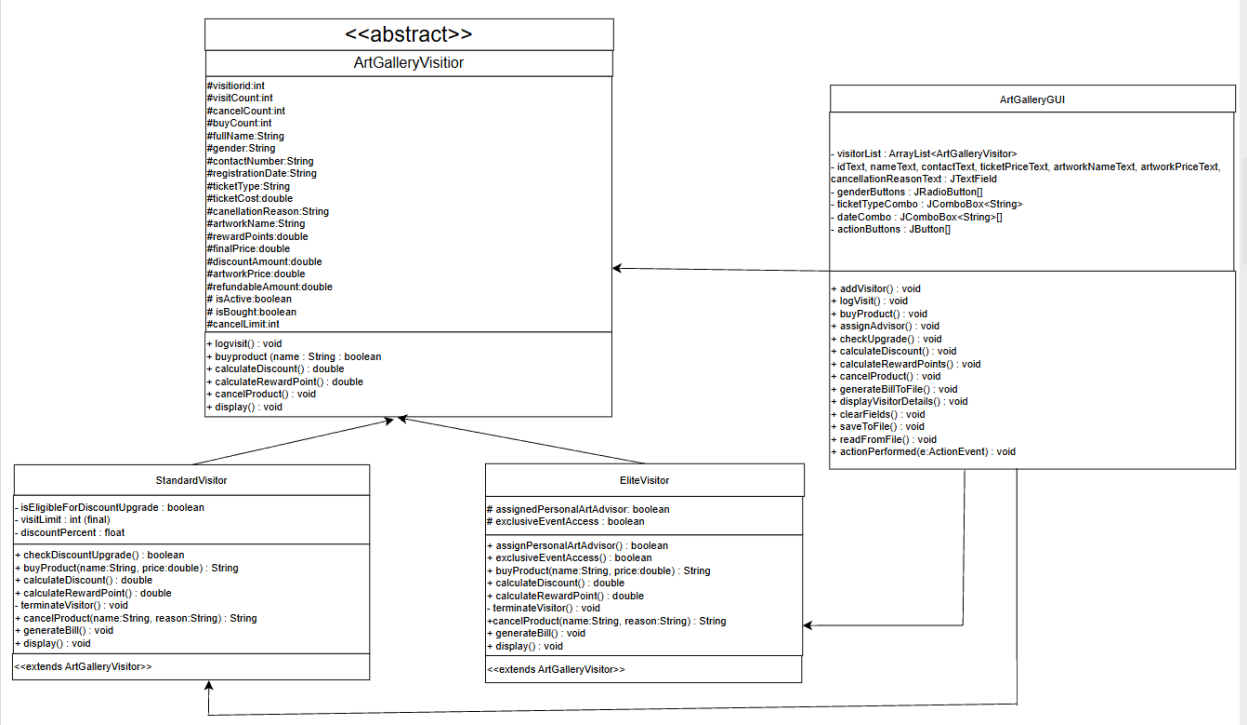


Figure 11: UML Class Diagram Representation

Pseudocode

Main GUI (ArtGalleryGUI)

Class ArtGalleryGUI extends JFrame implements ActionListener

Variables:

TextFields: idText, fullNameText, contactNumberText, ticketPriceText, artworkNameText, expensesValueText, withdrawalReasonsText

RadioButtons: maleBtn, femaleBtn, otherBtn

ComboBoxes: dayCombo, monthCombo, yearCombo, ticketTypeBox

Buttons: addVisitorBtn, logVisitBtn, buyProductBtn, assignAdvisorBtn, checkUpgradeBtn, calcDiscountBtn, calcRewardBtn, cancelProductBtn, billBtn, displayBtn, clearBtn, saveFileBtn, readFileBtn, exitBtn, eventAccessBtn

ArrayList visitorList

Constructor ArtGalleryGUI():

Setup font, title, size, close operation

Create mainPanel with sections:

Visitor Information

Visit Management

Purchase Management

Benefits & Rewards

File Operations

Others

Add input fields and buttons to each section

Register action listeners

Show GUI

Method actionPerformed(event):

TRY

IF event source == addVisitorBtn THEN call addVisitor()

ELSE IF == logVisitBtn THEN call logVisit()

ELSE IF == buyProductBtn THEN call buyProduct()

ELSE IF == assignAdvisorBtn THEN call assignAdvisor()

ELSE IF == checkUpgradeBtn THEN call checkUpgrade()

ELSE IF == eventAccessBtn THEN call checkEventAccess()

ELSE IF == calcDiscountBtn THEN call calculateDiscount()

ELSE IF == calcRewardBtn THEN call calculateRewardPoints()

ELSE IF == cancelProductBtn THEN call cancelProduct()

ELSE IF == billBtn THEN call generateBillToFile()

ELSE IF == displayBtn THEN call displayVisitorDetails()

ELSE IF == clearBtn THEN call clearFields()

ELSE IF == saveFileBtn THEN call saveToFile()

ELSE IF == readFileBtn THEN call readFromFile()

ELSE IF == exitBtn THEN EXIT PROGRAM

CATCH DuplicateVisitorIdException → Show warning

CATCH NumberFormatException → Show error

CATCH FileNotFoundException → Show error

CATCH IOException → Show error

CATCH Exception → Show general error

END Method

END Class

Abstract Class: ArtGalleryVisitor

Abstract Class ArtGalleryVisitor

Protected Variables:

visitorId, visitCount, cancelCount, buyCount

fullName, gender, contactNumber, registrationDate, ticketType

cancellationReason, artworkName

ticketCost, rewardPoints, finalPrice, discountAmount, artworkPrice, refundableAmount

cancelLimit = 3

isActive, isBought

Constructor(visitorId, fullName, gender, contactNumber, registrationDate, ticketPrice, ticketType):

Initialize all fields with default values

Method logVisit():

Increase visitCount

Mark visitor as active

Abstract Method buyProduct(name, price)

Abstract Method calculateDiscount()

Abstract Method calculateRewardPoint()

Abstract Method cancelProduct(name, reason)

Abstract Method generateBill()

Method display():

Print visitor details (id, name, visits, reward, bought)

END Class

Class: StandardVisitor

Class StandardVisitor extends ArtGalleryVisitor

Variables:

isEligibleForDiscountUpgrade

visitLimit = 5

discountPercent = 0.10

Constructor(...):

Call parent constructor

discountPercent = 10%

Method checkDiscountUpgrade():

IF visitCount >= visitLimit THEN

isEligibleForDiscountUpgrade = TRUE

discountPercent = 15%

Method buyProduct(name, price):

IF not isActive → RETURN "Log visit first"

IF already bought same artwork → RETURN "Already purchased"

Set artworkName, artworkPrice, isBought = TRUE

Increment buyCount

RETURN "Purchase successful"

Method calculateDiscount():

IF not isBought → RETURN 0

Call checkDiscountUpgrade()

discountAmount = artworkPrice * discountPercent

finalPrice = artworkPrice - discountAmount

RETURN discountAmount

Method calculateRewardPoint():

IF not isBought → RETURN 0

rewardPoints += finalPrice * 5

RETURN rewardPoints

Method cancelProduct(name, reason):

IF cancelCount >= cancelLimit → terminateVisitor() and RETURN "Account terminated"

IF no product OR name not match → RETURN "No product/cannot cancel"

Reset product info, set cancellation reason

refundableAmount = artworkPrice - 10% fee

rewardPoints -= finalPrice * 5

Increment cancelCount, decrement buyCount

RETURN "Cancelled. Refund: " + refundableAmount

Method generateBill():

IF not isBought → PRINT "No purchase"

ELSE print visitor + artwork + discount + final price

END Class

Class: EliteVisitor

Class EliteVisitor extends ArtGalleryVisitor

Variables:

assignedPersonalArtAdvisor

exclusiveEventAccess

Constructor(...):

Call parent constructor

advisor = FALSE, eventAccess = FALSE

Method assignPersonalArtAdvisor():

IF rewardPoints > 5000 → advisor = TRUE

RETURN advisor

Method exclusiveEventAccess():

IF advisor = TRUE → eventAccess = TRUE

RETURN eventAccess

Method buyProduct(name, price):

IF not isActive → RETURN "Log visit first"

IF already bought same artwork → RETURN "Already purchased"

Set artworkName, artworkPrice, isBought = TRUE

Increment buyCount

RETURN "Purchase successful"

Method calculateDiscount():

IF not isBought → RETURN 0

discountAmount = artworkPrice * 0.40

finalPrice = artworkPrice - discountAmount

RETURN discountAmount

Method calculateRewardPoint():

IF not isBought → RETURN 0

rewardPoints += finalPrice * 10

RETURN rewardPoints

Method cancelProduct(name, reason):

IF cancelCount >= cancelLimit → terminateVisitor() and RETURN "Account terminated"

IF no product OR name not match → RETURN "No product/cannot cancel"

Reset product info, cancellation reason

refundableAmount = artworkPrice - 5% fee

rewardPoints -= finalPrice * 10

Increment cancelCount, decrement buyCount

RETURN "Cancelled. Refund: " + refundableAmount

Method generateBill():

IF not isBought → PRINT "No purchase"

ELSE print visitor + artwork + discount + final price

END Class

Method Description

Class: ArtGalleryVisitor (Abstract Parent)

- **logVisit()**
  This function logs a visitor's arrival at the gallery. It increments the visitCount by one and marks the isActive flag true, which means the visitor is active at present.

- **buyProduct (String name, double price)** *(abstract)*
  To be overridden by subclasses. Stands for the purchase of a piece of art by saving art information and modifying purchase-related properties.

- **calculateDiscount()** *(abstract)*
  Subclass-level implementation to calculate and return the discount value depending on the type of visitor.

- **calculateRewardPoint()** *(abstract)*
  Computes reward points earned from a purchase. It is calculated depending on the visitor type and purchase value.

- **cancelProduct (String name, String reason)** *(abstract)*
  Handles cancellation of a product bought, updates cancellation reason, calculates refund, and processes cancellation count.

- **generateBill()** *(abstract)*
  Subclass-specific implementation to output a formatted bill showing visitor details, purchase details, discount, and final price.

- **display()**
  Prints fundamental visitor details (ID, name, visit count, reward points, and purchase status) to the console.

Class: StandardVisitor (Subclass of ArtGalleryVisitor)

- **checkDiscountUpgrade()**

  Checks whether the total visits made by the visitor exceed the given limit (visitLimit = 5). If yes, the visitor is eligible for a discount upgrade, and the discount percentage goes up from 10% to 15%. Returns boolean to validate upgrade.

- **buyProduct (String name, double price)**

  Allows a visitor to purchase art. First, ensures the visitor is active (should log visit before purchasing). Disallows the purchase of the same items if the same item was bought earlier. Saves artwork data on success, sets the isBought flag to true, and increases the buyCount.

- **calculateDiscount()**

  Computes the value of discount on the artwork bought. It also checks whether the visitor has experienced a discount upgrade. Price is calculated after discount application.

- **calculateRewardPoint()**

  Creates reward points for a Standard visitor by returning the final purchase price multiplied by 5. Reward points earned are returned.

- **terminateVisitor() (private)**

  An auxiliary function to close the account of a visitor in case of excessive cancellation. It resets activity status, reward points, and visit counts.

- **cancelProduct (String name, String reason)**

  Cancel an artwork purchased earlier if valid. If cancellation limit reached, account is closed. Otherwise, product is removed, refund (90% of price of artwork) given, reward points deducted, and number of cancellations increased.

- **generateBill()**

Display a detailed bill in console for visitor's purchase, including visitor details, artwork details, discount applied, and amount paid.

- **display()**
  Extends the parent's display mode to show the discount upgrade status and the existing discount percentage.

Class: EliteVisitor (Subclass of ArtGalleryVisitor)

- **personalArtAdvisor()**
  Checks if the visitor has greater than 5000 reward points. If so, a personal art advisor is assigned and the method returns true.

- **exclusiveEventAccess()**
  Grants exclusive event access to an Elite visitor in the event of an assigned personal advisor. Returns true on availability.

- **buyProduct (String name, double price)**
  Like Standard visitor purchase, but allows the visitor to buy an artwork of higher benefits. Prevents multiple buys and updates artwork data.

- **calculateDiscount()**
  Calculates 40% discount from artwork price for Elite visitors and updates final price.

- **calculateRewardPoint()**
  Elite visitors get reward points 10 times final price. Points are added on each buy.

- **terminateVisitor()**
  Deactivates an Elite visitor beyond the cancellation limit. Resets event access and advisor assignment too.

- **cancelProduct (String name, String reason)**
  Cancels a purchase and refunds 95% of the cost of the artwork. For cancellations over the limit, closes the account. Reward points are proportionally deducted.

- **generateBill()**
  Prints a printed bill to the console, similar to Standard visitors but with Elite privileges added.

- **display()**
  Shows visitor's overall information from parent class, including advisor assignment and special event attendance status.

Class: ArtGalleryGUI

- **addVisitor()**
  Collects input data from form fields and adds a new visitor (Standard or Elite). Checks for existing IDs and adds the visitor to ArrayList.

- **logVisit()**
  Finds a visitor by ID and invokes the logVisit() method to increment their visit count and enable them.

- **buyProduct()**
  Gets visitor by ID, gets artwork details (name and price) from GUI fields, and calls visitor's buyProduct() method. Prints success/error message.

- **personalArtAdvisor()**
  Assigns a personal art advisor to an Elite visitor if eligible. Prints success or failure messages.

- **checkUpgrade()**

Calls checkDiscountUpgrade() for Standard visitors to ascertain if they are eligible for upgraded discount benefits.

- **checkEventAccess()**
  Grants special event access to Elite visitors if already assigned a personal art advisor.

- **calculateDiscount()**
  Calls visitor's calculateDiscount() method and prints discount amount.

- **calculateRewardPoints()**
  Calls visitor's calculateRewardPoint() method and prints reward points.

- **cancelProduct()**
  Cancels a visitor's purchase of a particular reason, increases cancellation count, and prints refund amount.

- **generateBill()**
  Calls visitor's generateBill() method and additionally writes the bill to a .txt file. The bill is printed in a dialog box for improved readability.

- **displayVisitorDetails()**
  Shows all visit records in a JTable with ID, name, gender, contact, ticket type, price, visits, purchases, and cancellations columns.

- **clearFields()**
  Resets all form input fields and drop-down menu choices to default values.

- **saveToFile()**
  Writes all visitor details from the ArrayList to VisitorDetails.txt in the table format.

- **readFromFile()**

  Reads VisitorDetails.txt and displays its content in a read-only text area of a dialog.

- **exit()**

  This button immediately closes the application when clicked. It calls System.exit(0), which terminates the running program and releases resources.

Test Cases and Results

Test Case 1

Compile and Run Program	
Objective	To verify that the program compiles and runs successfully.
Action	Open Command Prompt (or terminal), navigate to your project folder, and run javac *.java and then java ArtGalleryGUI.
Expected Output	Program compiles without errors and GUI window opens.
Actual Output	The program run and compile successfully and GUI windows open.
Result	The test was successful.

Table 1:Test-1: Compile and Run program.

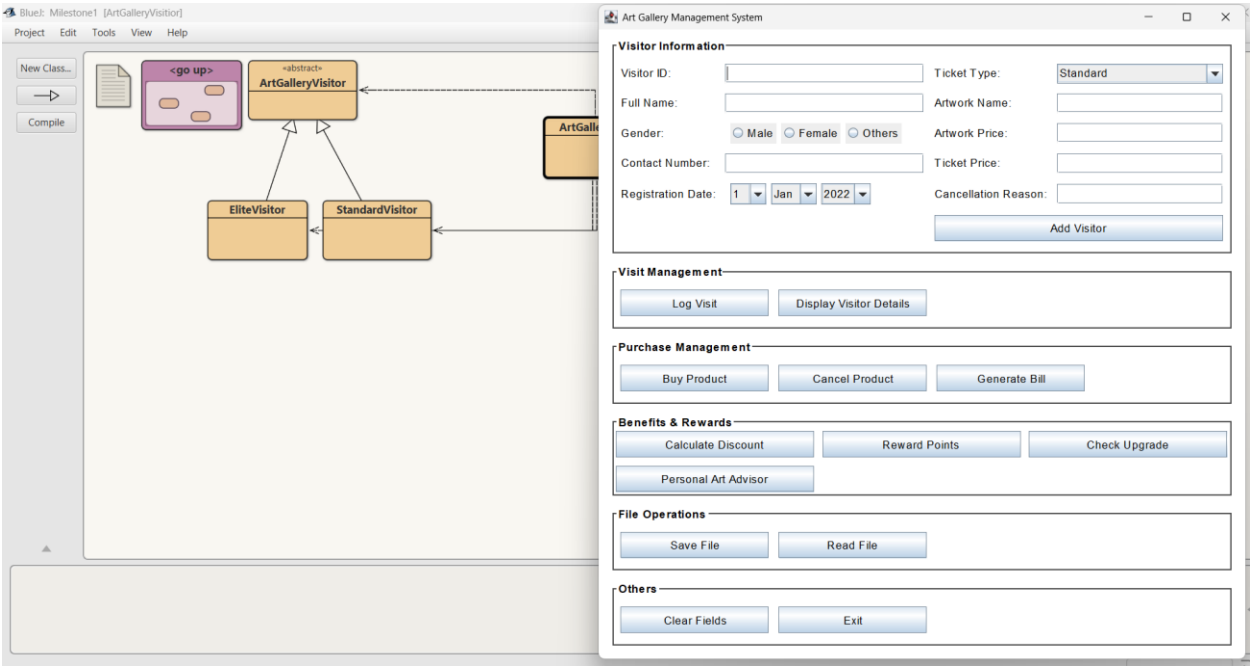


Figure 12: Test-1: Compile and Run program

Test Case 2

Adding Standard and Elite Visitors	
Objective	To check if both Standard and Elite visitors can be added.
Action	Fill all required fields in GUI and then Select “Standard”, Click Add Visitor. Repeat for “Elite”.
Expected Output	Message dialog “Visitor Added Successfully!”. Visitor should be stored in ArrayList.
Actual Output	The message dialog must show “Visitor Added Successfully!”.
Result	The test was successful.

Table 2: Test-2: Standard and Elite visitors can be added.

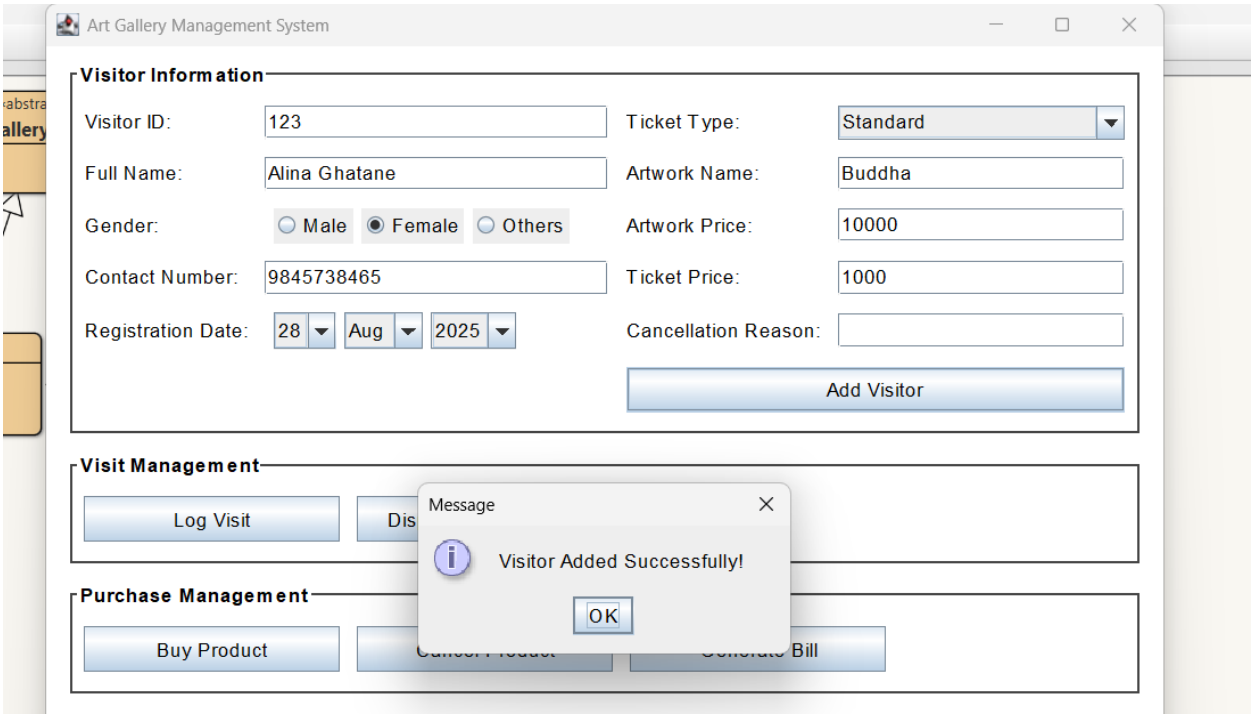


Figure 13: Test-2: Adding StandardVisitor

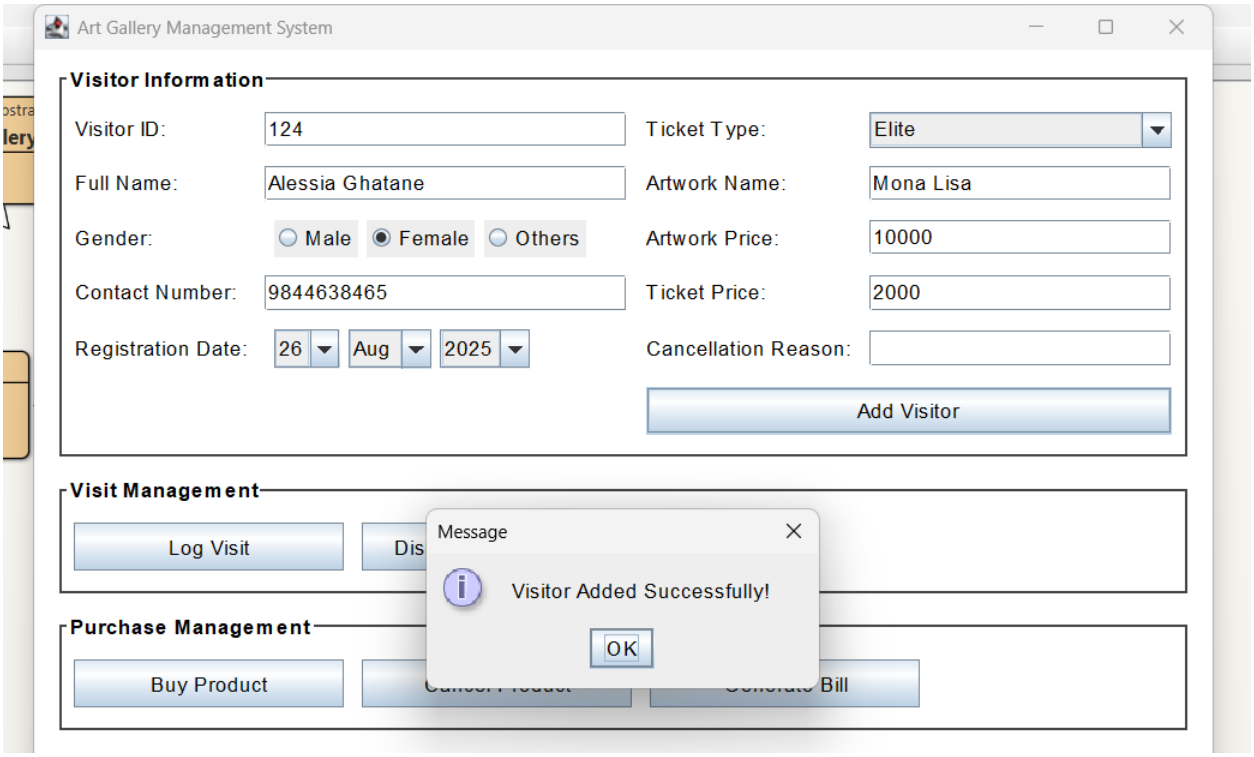


Figure 14: Test-2: Adding EliteVisitor

Test Case 3: Log Visit, Buy Product, Cancel Visit

Log Visit	
Objective	To verify that a visitor's visit can be logged successfully and the visit count increases.
Action	Enter an existing Visitor ID in the text field, Click Log Visit button.
Expected Output	Message dialog appears: "Visit logged. Total visits: X" (X = incremented count). Visitor's visitCount increases, and isActive becomes true.
Actual Output	Message dialog displayed: "Visit logged. Total visits: 1" (or the correct updated count).
Result	The test was successful.

Table 3: Test-3: Logged successfully.

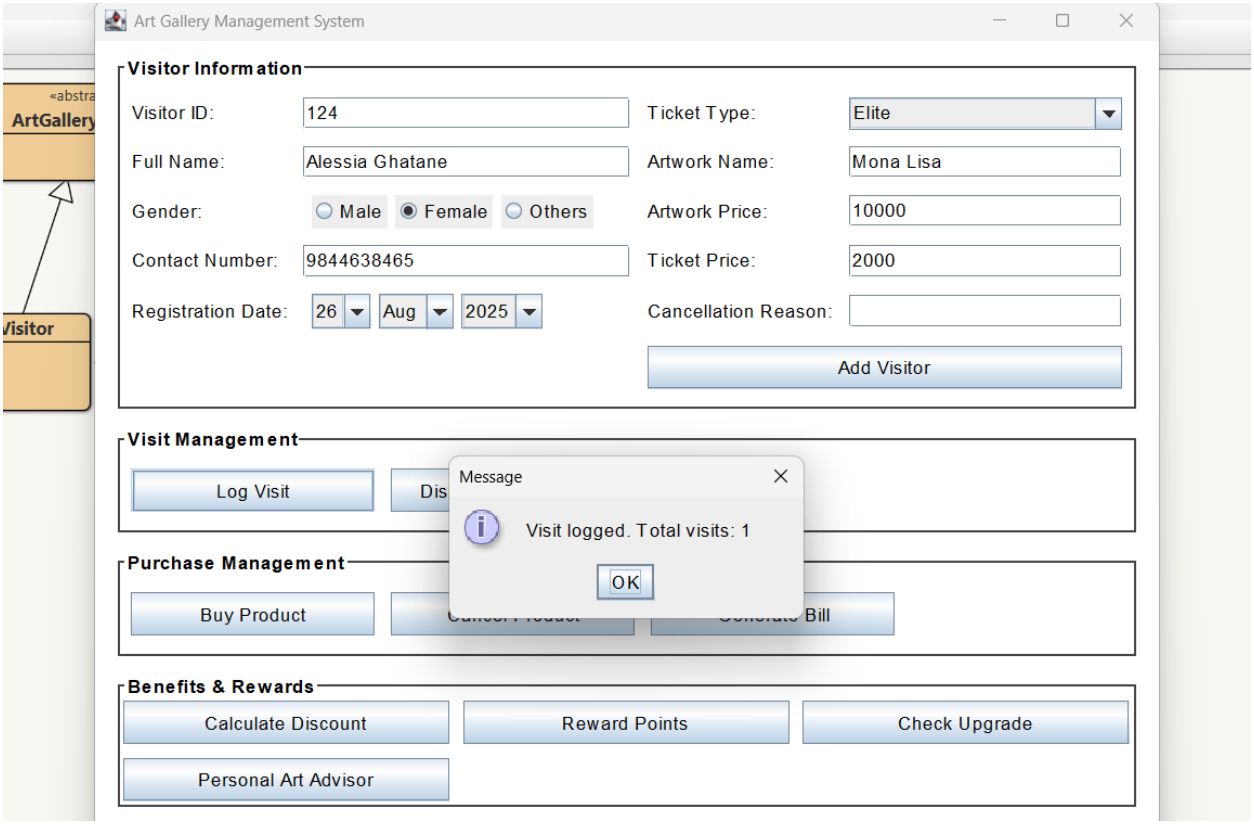


Figure 15: Test-3: Logged successfully

Buy Product	
Objective	To verify that a visitor can purchase an artwork after logging a visit.
Action	Enter Visitor ID , Enter artwork name and price, Click Buy Product button.
Expected Output	Message dialog: “Purchase successful.” Artwork name and price are stored, isBought = true, buyCount increments by 1.
Actual Output	Message dialog displayed: “Purchase successful.” Artwork stored successfully in visitor record.
Result	The test was successful.

Table 4: Test-3: Purchase successfully.

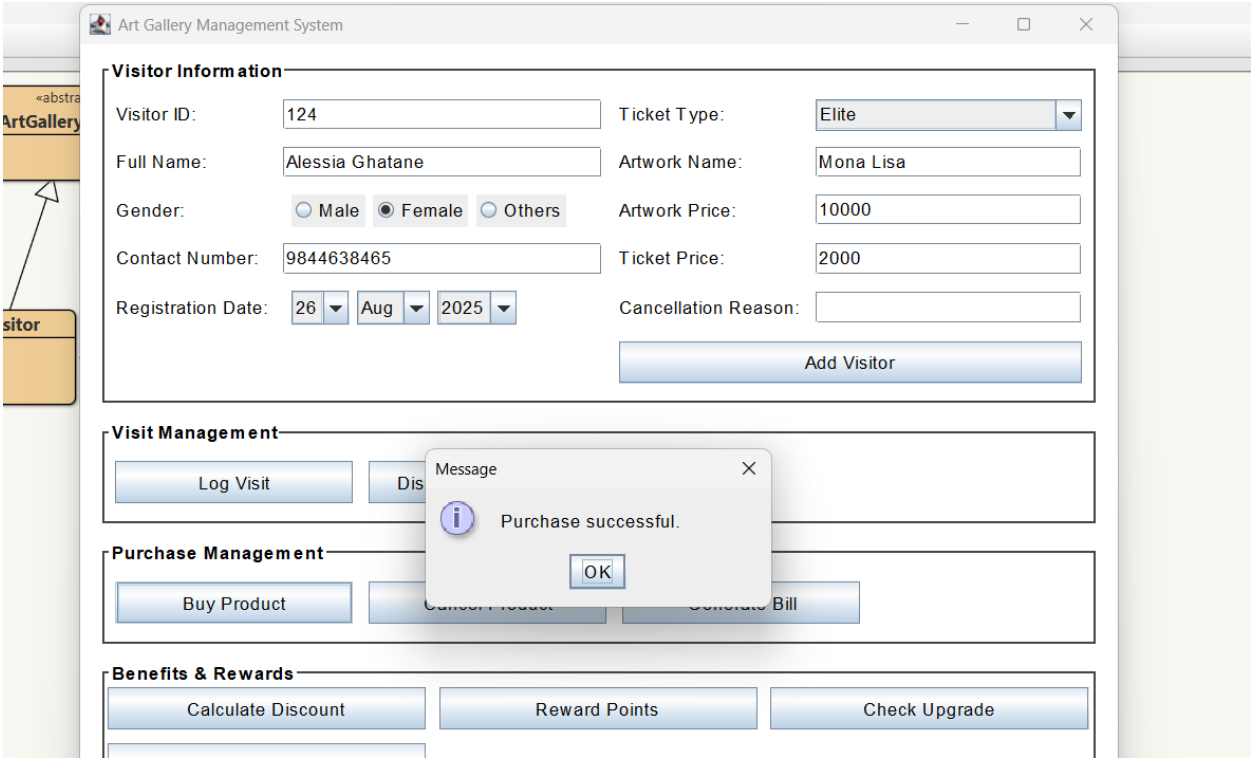


Figure 16: Test-3: Purchase successfully

Cancel Product	
Objective	To verify that a purchased artwork can be cancelled with refund calculation.
Action	Enter Visitor ID, Enter the same artwork name, Enter a cancellation reason, Click Cancel Product button.
Expected Output	Message dialog: “Cancelled. Refund: XXXX.XX”. Refund calculated correctly (Standard = 90% refund, Elite = 95%). Artwork reset (isBought = false, artworkName = null), cancelCount increments.
Actual Output	Message dialog displayed with refund details, e.g., “Cancelled. Refund: 900.0”. Visitor’s cancellation recorded successfully.
Result	The test was successful.

Table 5: Test-3: Cancelled with Refund.

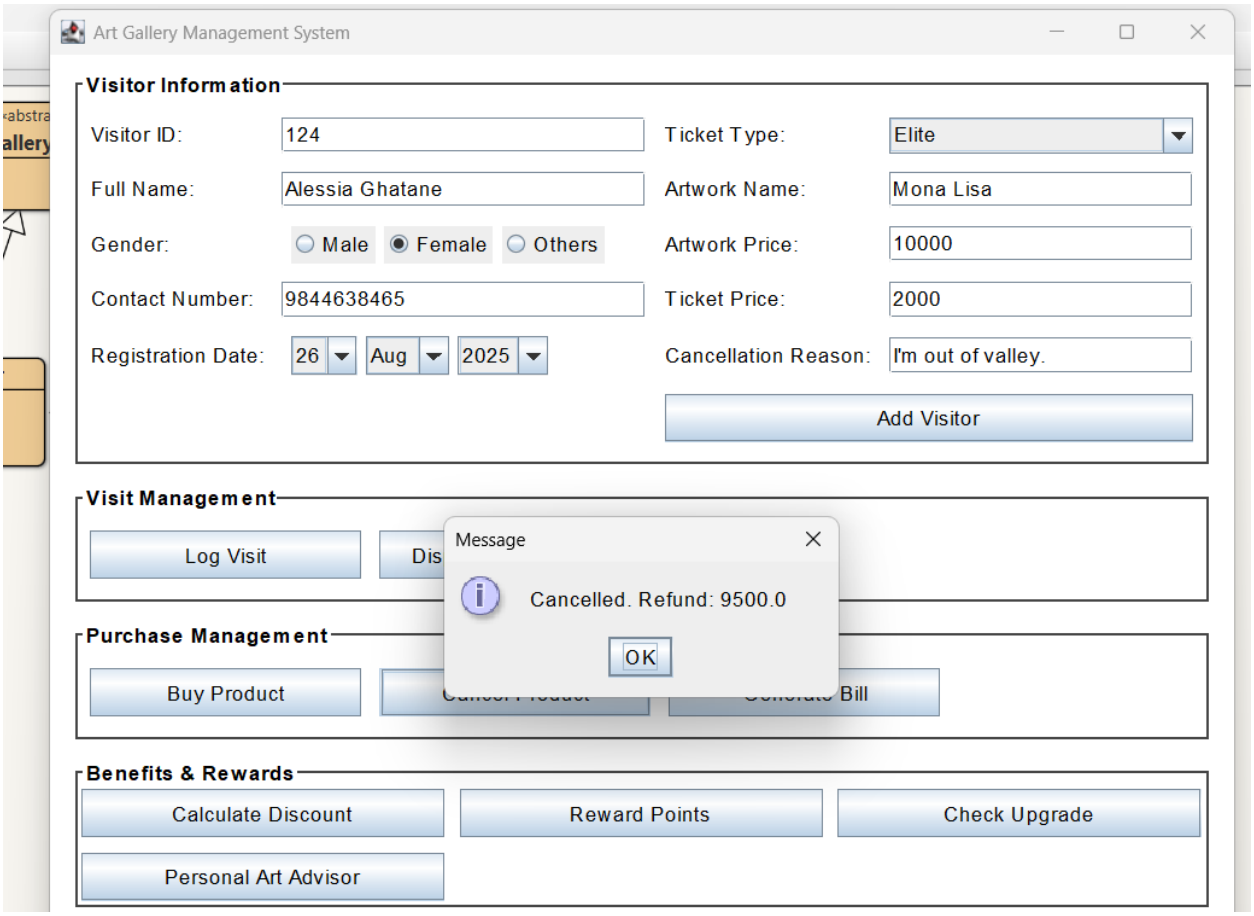


Figure 17: Test-3: Cancel product

Test Case 4: Check Upgrade, Calculate Discount and Calculate Reward Points

Check Upgrade (Standard Visitor)	
Objective	To verify that a Standard visitor receives a discount upgrade after 5 visits.
Action	Added a Standard visitor, Logged visit 5 times, Entered Visitor ID, Clicked Check Upgrade button.
Expected Output	A message dialog: “Upgrade achieved! Discount increased.” Also, discountPercent should change from 0.10 to 0.15.
Actual Output	The system displayed: “Upgrade achieved! Discount increased.” DiscountPercent updated to 0.15.
Result	The test was successful.

Table 6: Test-4: Discount upgrade (Standard visitor).

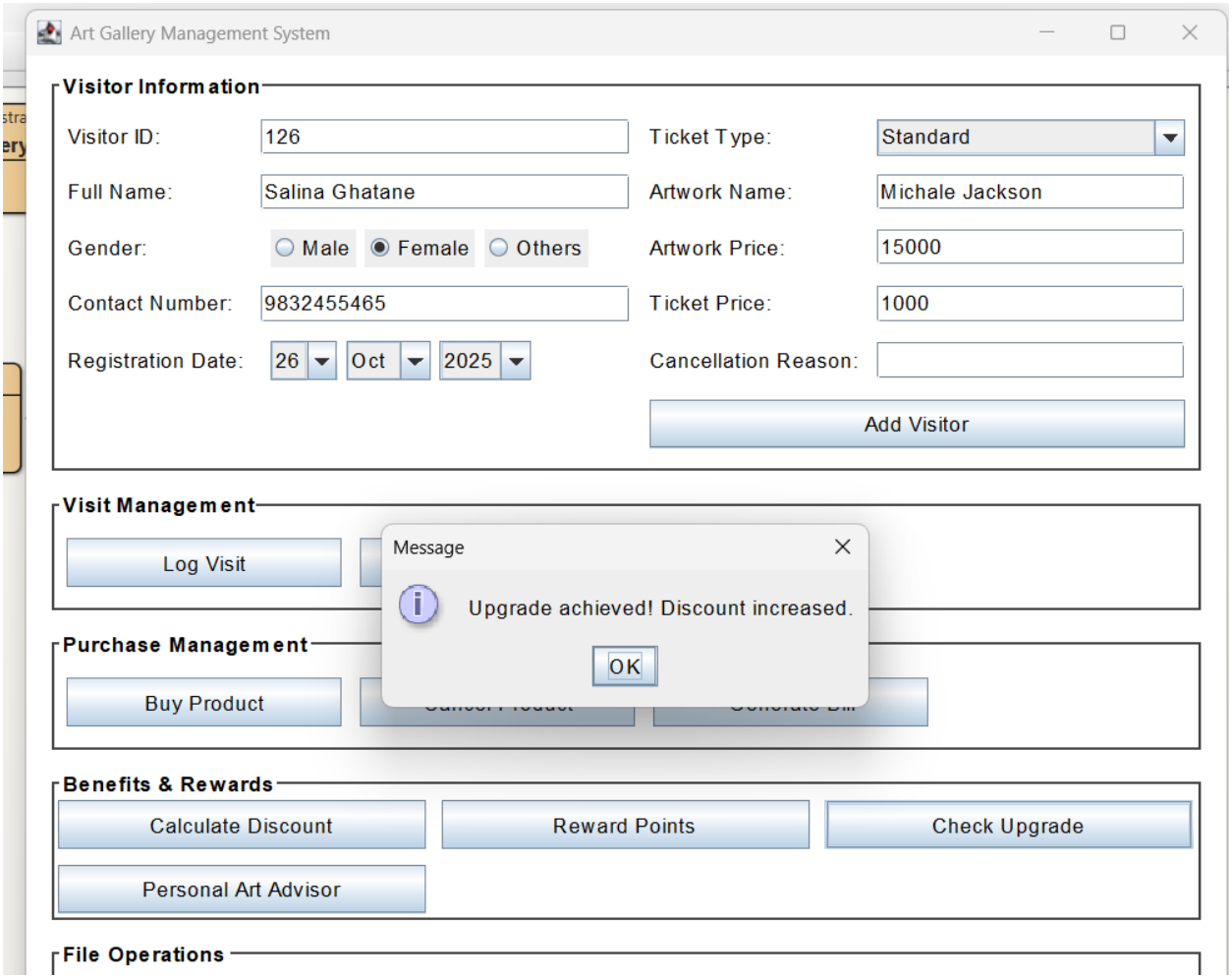


Figure 18: Test-4: Check upgrade (StandardVisitor)

Calculate Discount (Standard Visitor)	
Objective	To verify that discount is calculated correctly for a Standard visitor.
Action	Added Standard visitor, Logged visit, Bought artwork worth 2000, Entered Visitor ID, Clicked Calculate Discount.
Expected Output	Discount = 2000 × 0.10 = 200. Final Price = 2000 – 200 = 1800. (If upgraded: discount = 2000 × 0.15 = 300, final price = 1700). Message dialog should display “Discount calculated: …”.

Actual Output	The system displayed “Discount calculated: 200.00” (before upgrade). After upgrade, it displayed “Discount calculated: 300.00”.
Result	The test was successful.

Table 7: Test-4: Calculate discount (Standard visitor).

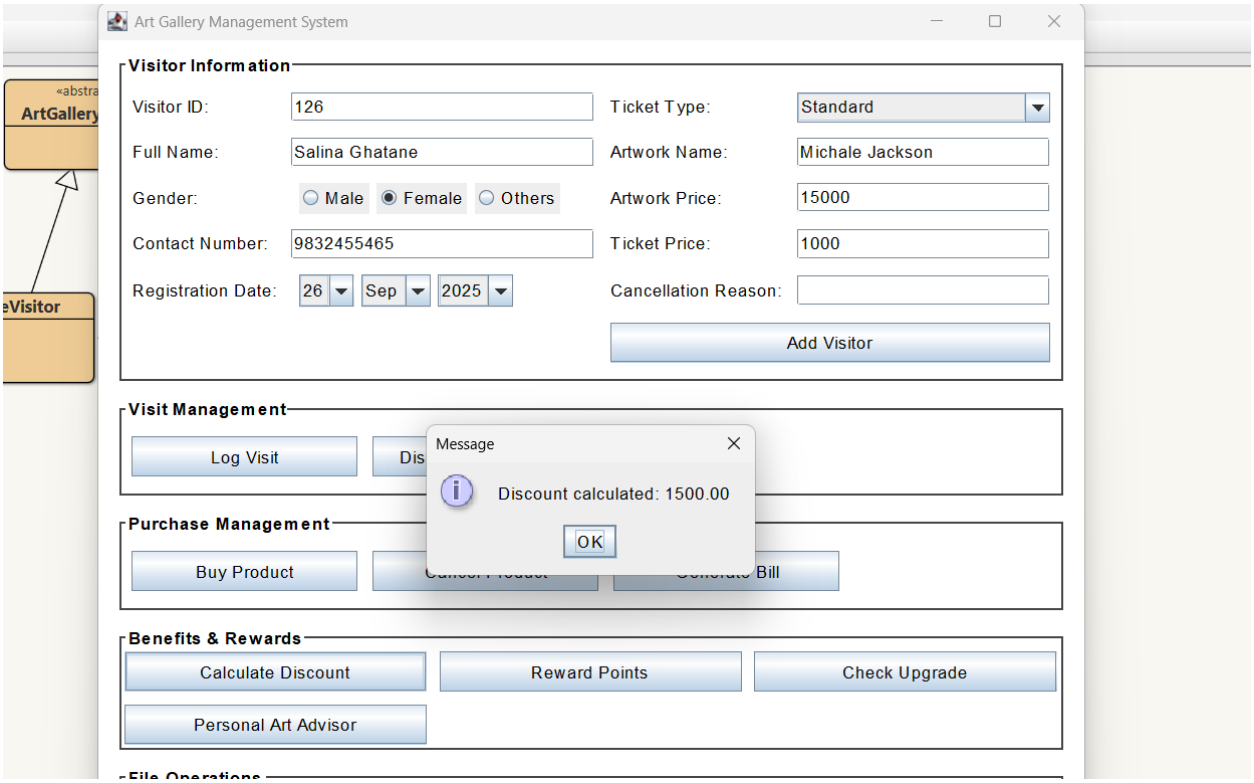


Figure 19: Test-4: Calculate Discount (Standard visitor)

Calculate Discount (Elite Visitor)	
Objective	To verify that discount is calculated correctly for an Elite visitor.
Action	Added Elite visitor, Logged visit, Bought artwork worth 5000, Entered Visitor ID, Clicked Calculate Discount.
Expected Output	Discount = 5000 × 0.40 = 2000. Final Price = 5000 – 2000 = 3000. Message dialog should display “Discount calculated: 2000.00”.
Actual Output	The system displayed “Discount calculated: 2000.00”.
Result	The test was successful.

Table 8v: Test-4: Calculate discount (Elite visitor).

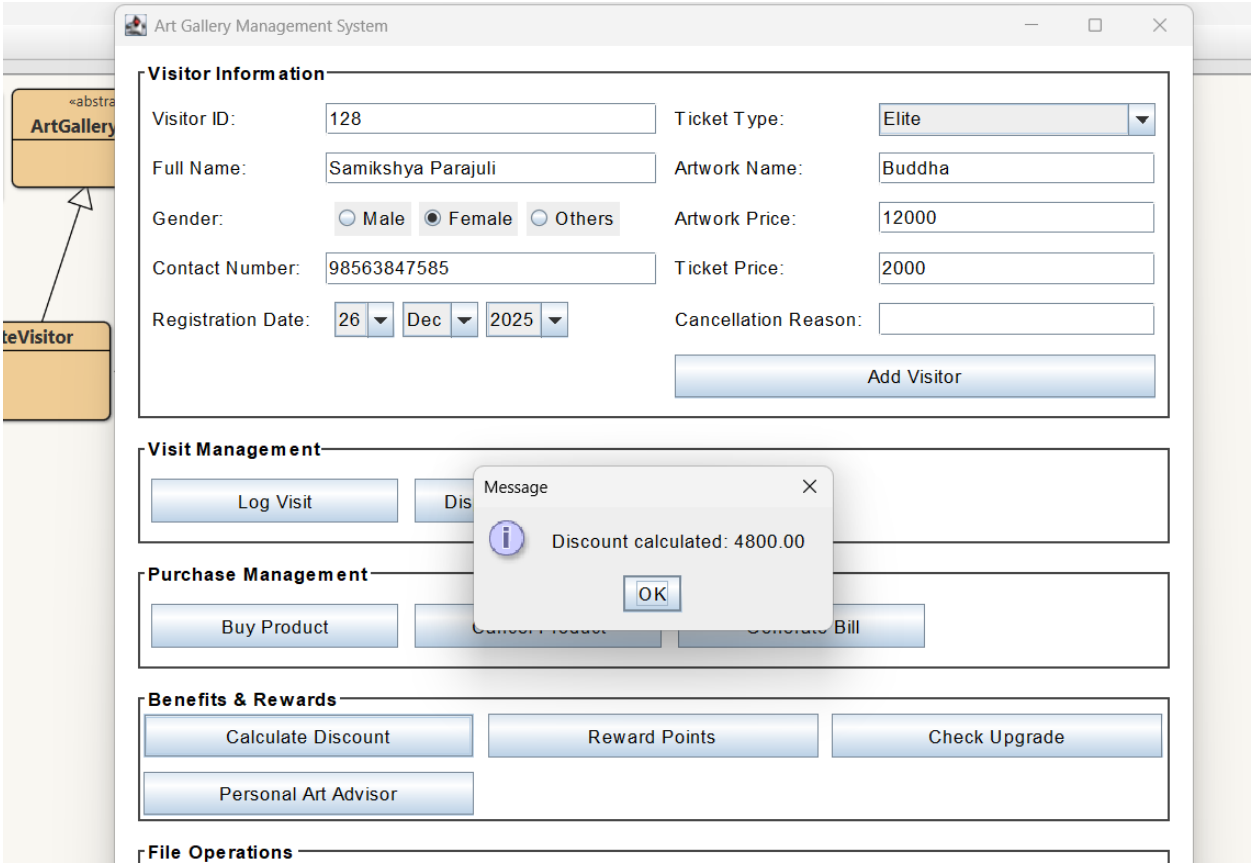


Figure 20: Test-4: Calculate Discount (EliteVisitor)

Calculate Reward Points (Standard Visitor)	
Objective	To verify that reward points are calculated correctly for a Standard visitor.
Action	Added Standard visitor, Logged visit, Bought artwork worth 2000 (discounted final price = 1800), Entered Visitor ID, Clicked Reward Points button.
Expected Output	Reward Points = 1800 × 5 = 9000 (if upgraded: 1700 × 5 = 8500). A message dialog should display “Reward Points: 9000.00”.
Actual Output	The system displayed “Reward Points: 9000.00”. After upgrade, displayed “Reward Points: 8500.00”.
Result	The test was successful.

Table 9: Test-4: Calculate discount (Standard visitor).

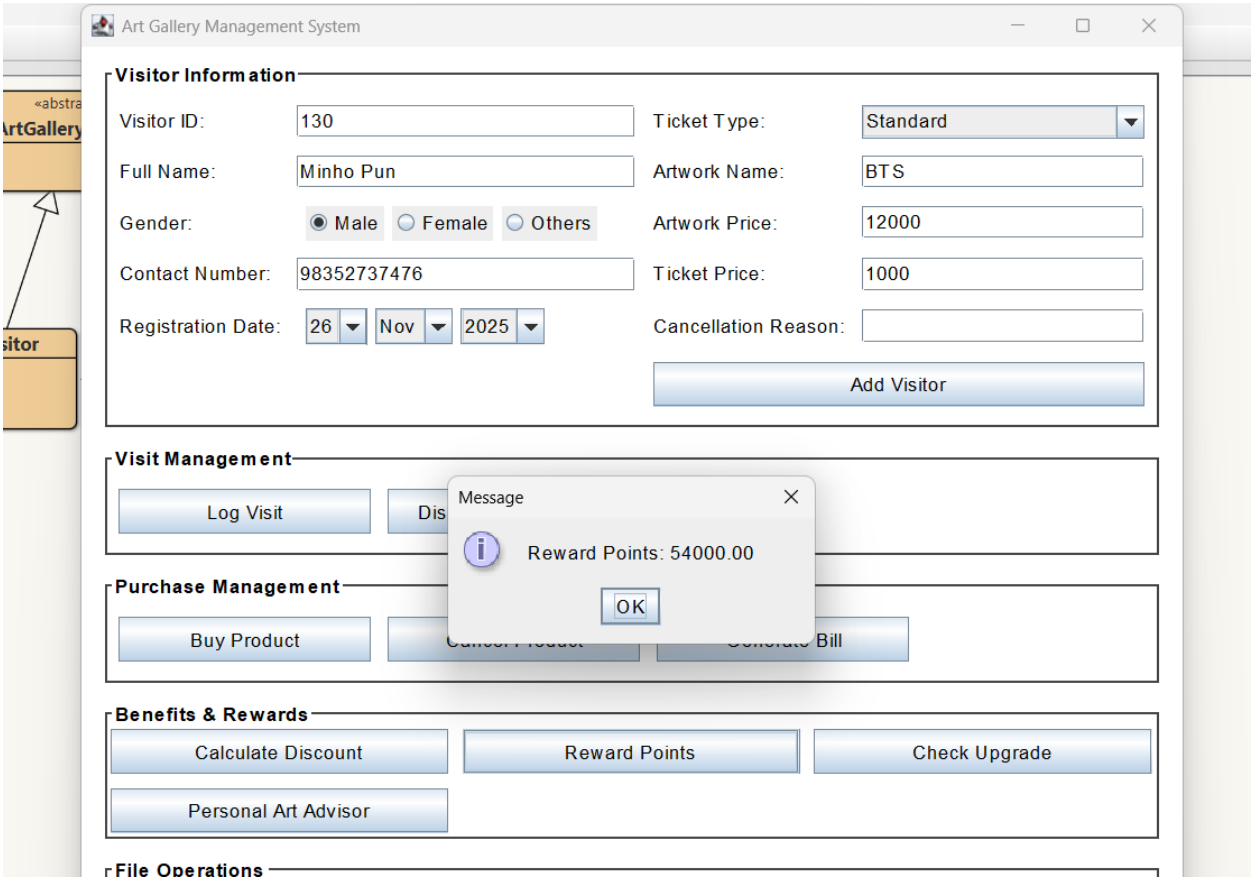


Figure 21: Test-4: Calculate Reward Points (StandardVisitor)

Calculate Reward Points (Elite Visitor)	
Objective	To verify that reward points are calculated correctly for an Elite visitor.
Action	Added Elite visitor, Logged visit, Bought artwork worth 5000 (final price = 3000), Entered Visitor ID, Clicked Reward Points button.
Expected Output	Reward Points = 3000 × 10 = 30000. A message dialog should display “Reward Points: 30000.00”.
Actual Output	The system displayed “Reward Points: 30000.00”.
Result	The test was successful.

Table 10: Test-4: Reward points (Elite visitor).

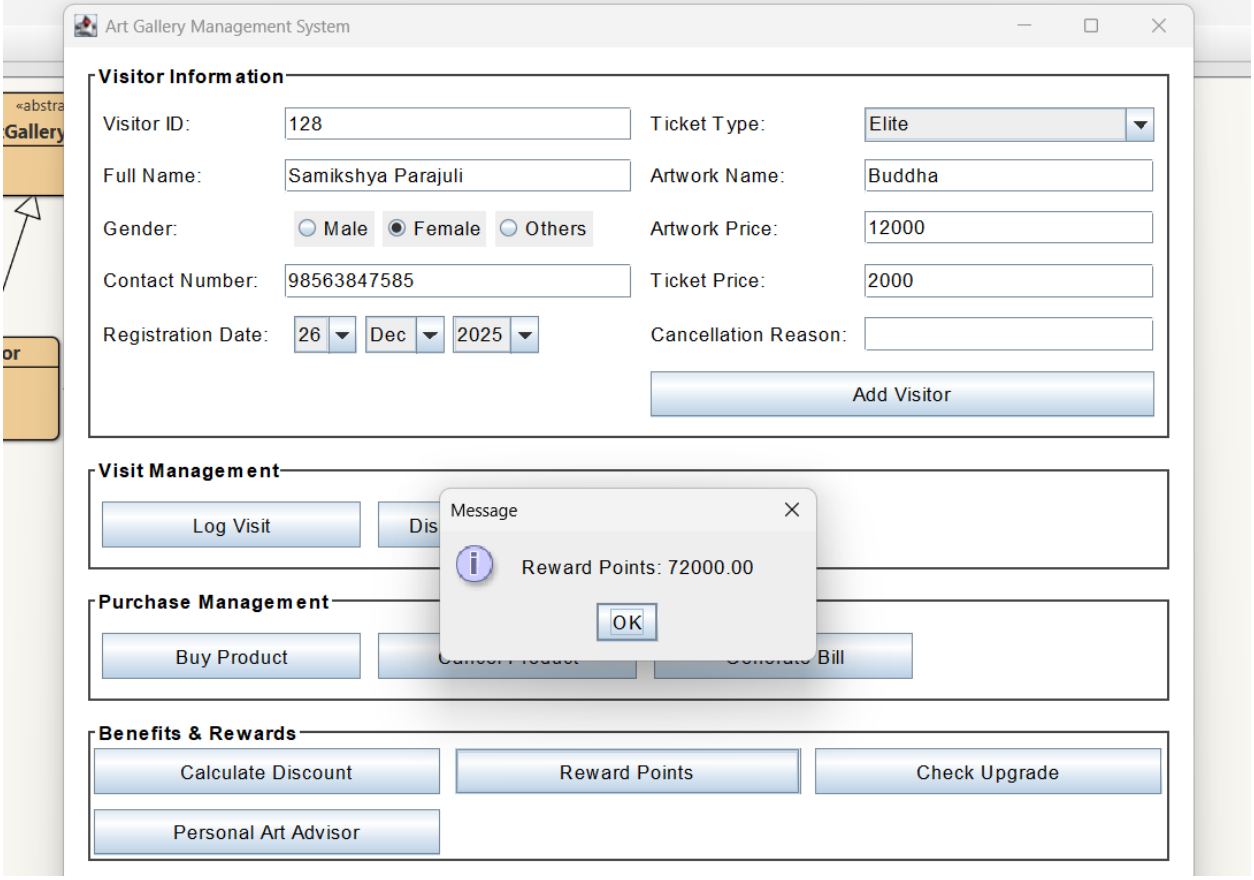


Figure 22: Test-4: Calculate Reward Points (EliteVisitor)

Test 5: Test case for saving and reading from a file.

Saving from a file	
Objective	To check if all visitor details can be saved into a file.
Action	After adding visitors, clicked Save File button.
Expected Output	Visitor details should be saved in VisitorDetails.txt. A success message should be shown: "Visitor details saved to VisitorDetails.txt".
Actual Output	File VisitorDetails.txt created successfully, contains all visitor details in formatted table, message dialog displayed.
Result	Test passed.

Table 11: Test-5: Saving a file.

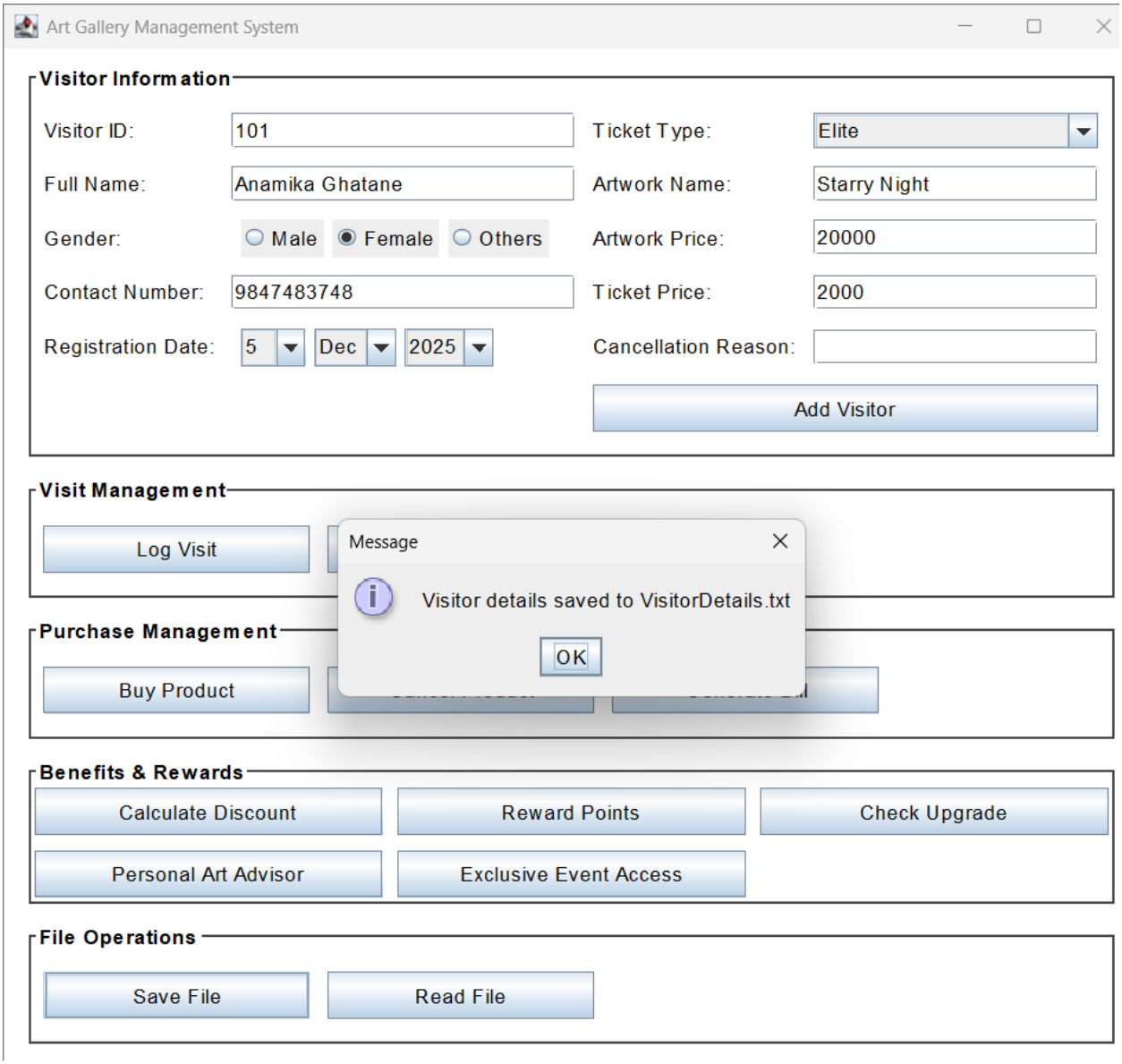


Figure 23: Test-5: Saving

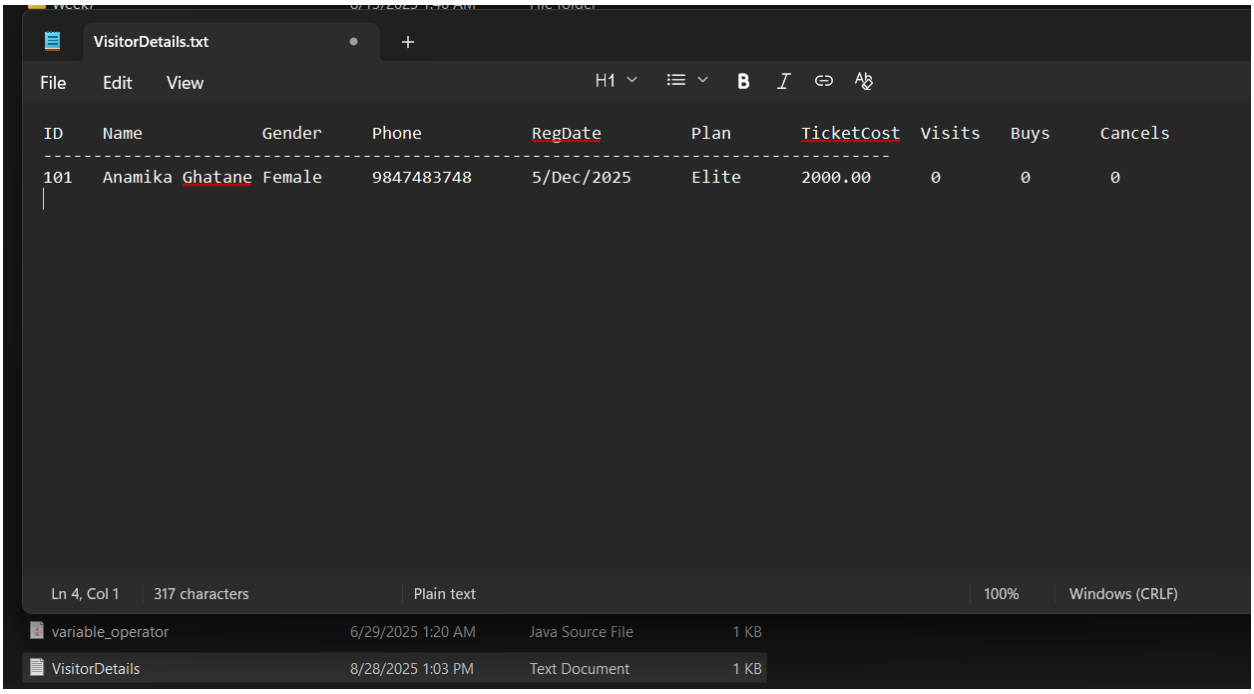


Figure 24: Test-5: VisitiorDetails.txt in Notepad

Reading from a file	
Objective	To verify if saved visitor details can be read back into the program
Action	Clicked Read File button.
Expected Output	Contents of VisitorDetails.txt should be read and displayed in a text area/dialog inside the GUI.
Actual Output	Visitor details successfully read from file and displayed in GUI dialog box.
Result	Test passed.

Table 12: Test-5: Reading from a file.

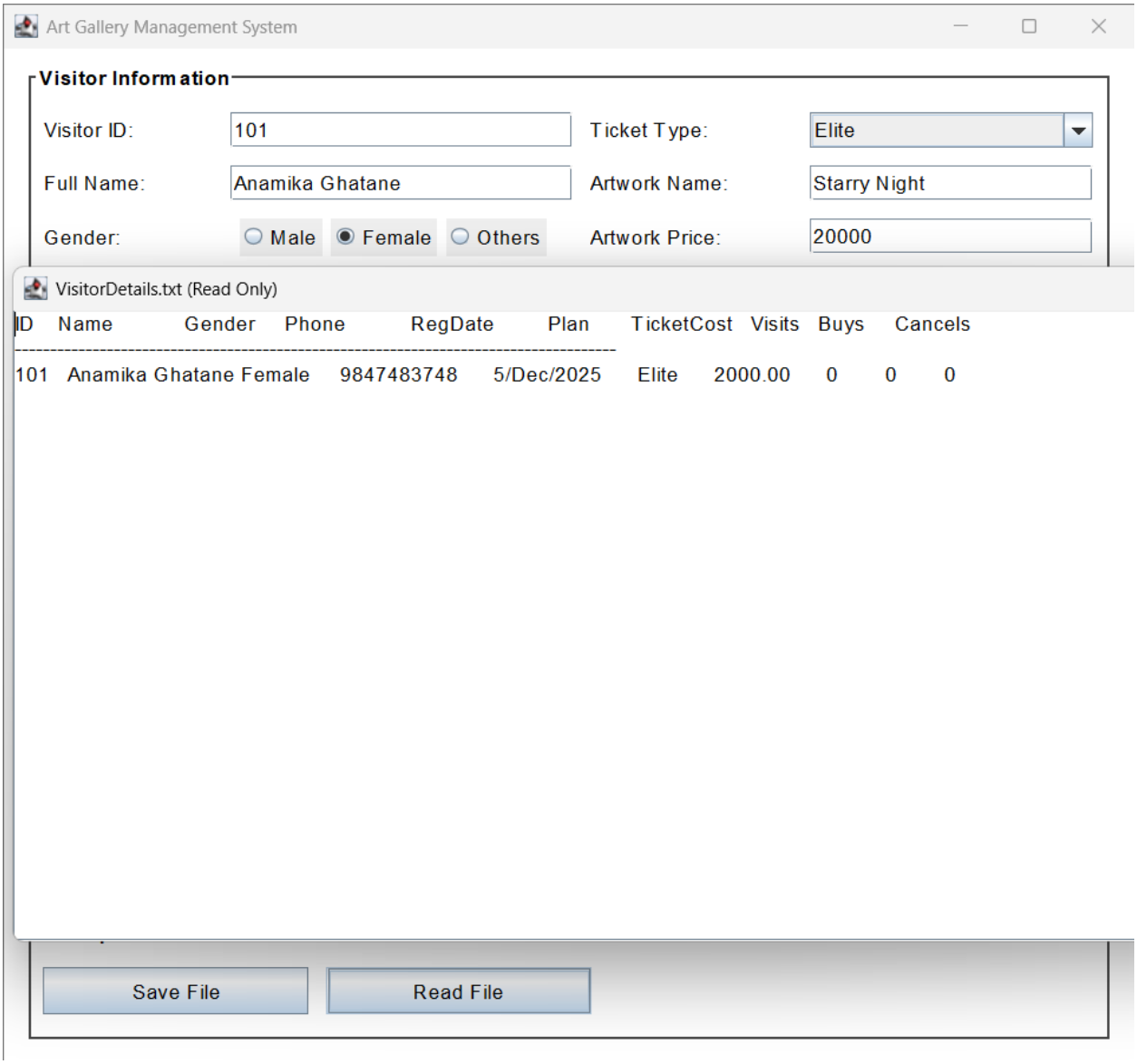


Figure 25: Test-5: Read from a file

.

Error Detection & Correction

Syntax error

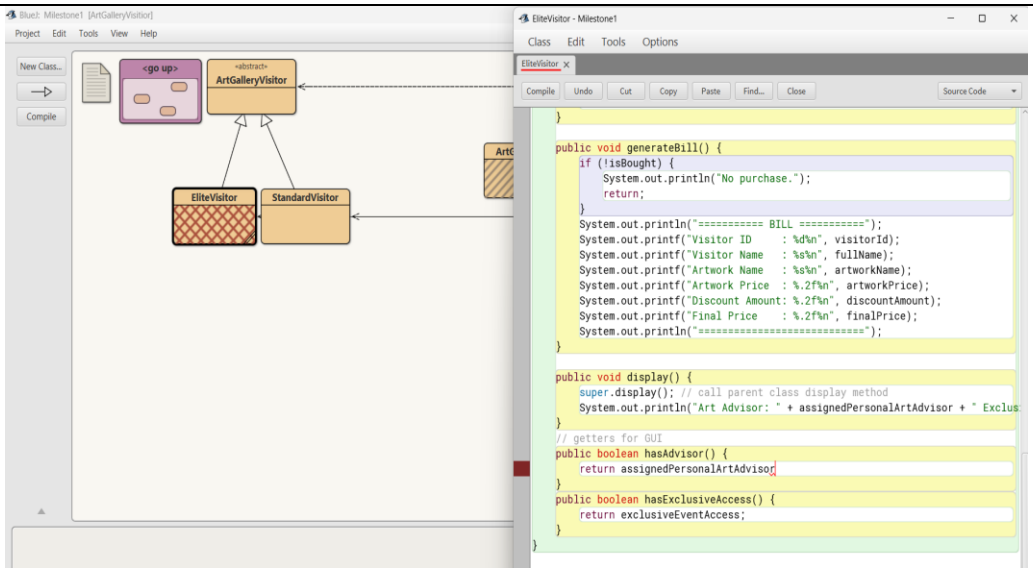
Syntax Error	
Description	In compiling EliteVisitor.java, I had left out a semicolon after a statement.
Screenshot	

Figure 26: Syntax Error on coding

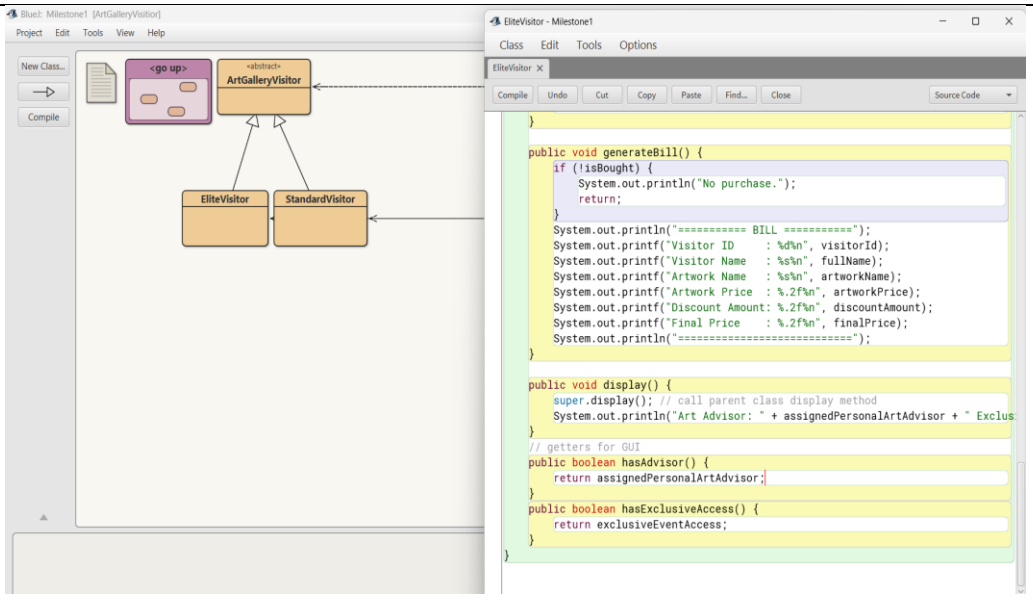
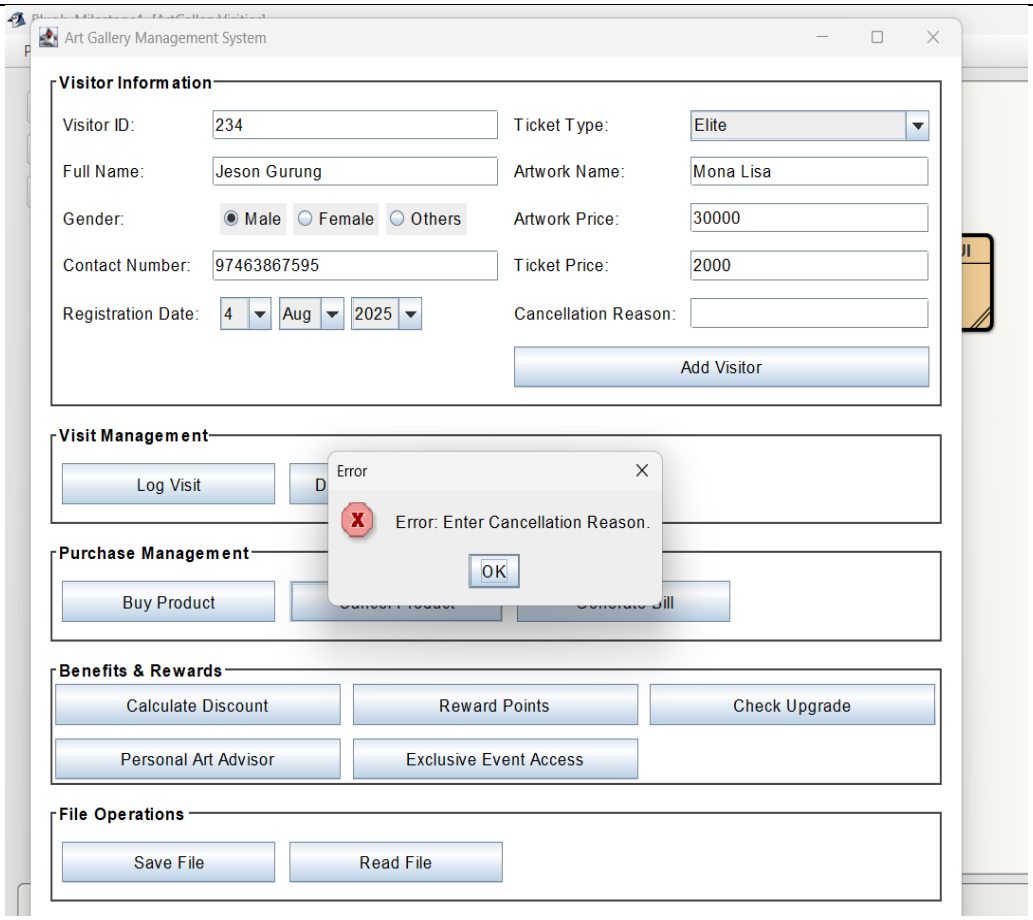
Correction	Added the semicolon.
Screenshot	

Figure 27: Problem Solved

Reflection	Syntax errors are pointed out by compiler, hence easy to correct.

Table 13: Syntax error and solved.

Semantic/Runtime error

Runtime Error	
Description	When clicking on "Cancel Product" without previously buying anything.
Screenshot	

Correction	Added condition within cancelProduct():
	
	`if (!isBought || !name.equals(this.artworkName)) {`
	`  return "No product/cannot cancel.";`
	`}`

Figure 28: Runtime Error

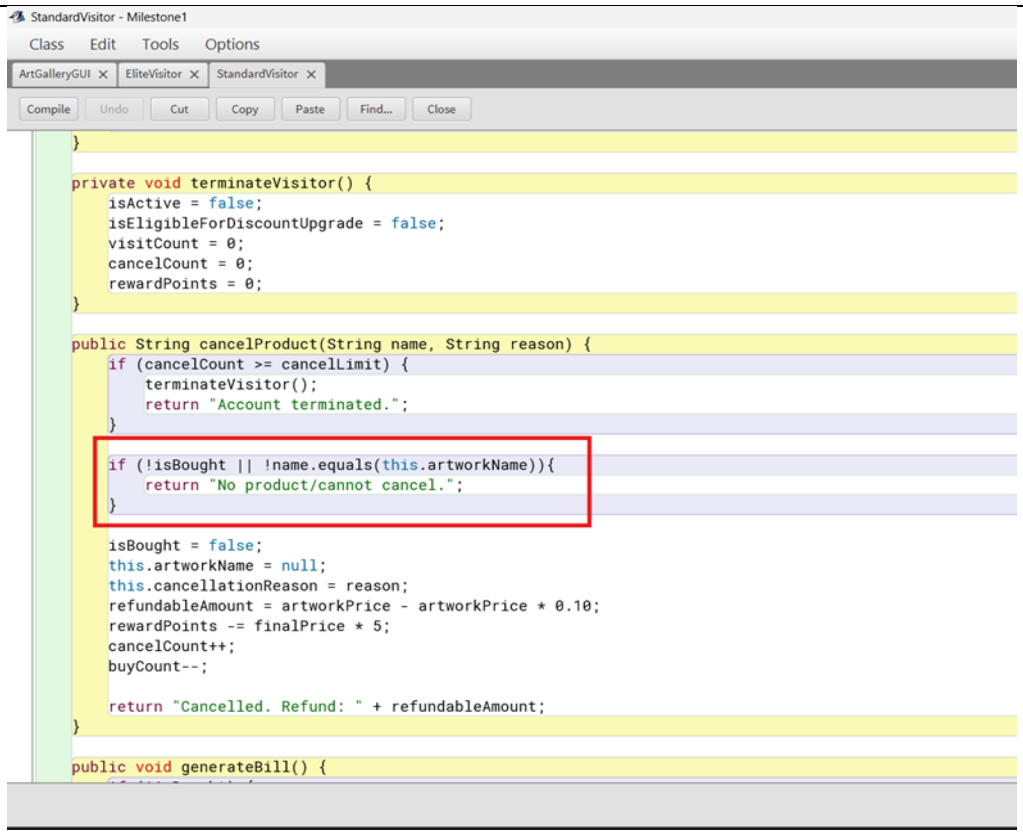
Screenshot	
Reflection	Runtime errors are when the wrong object/state is accessed. Corrected by adding proper checks.

Figure 29: Correction code

Table 14: Runtime error and correction code.

Logical error

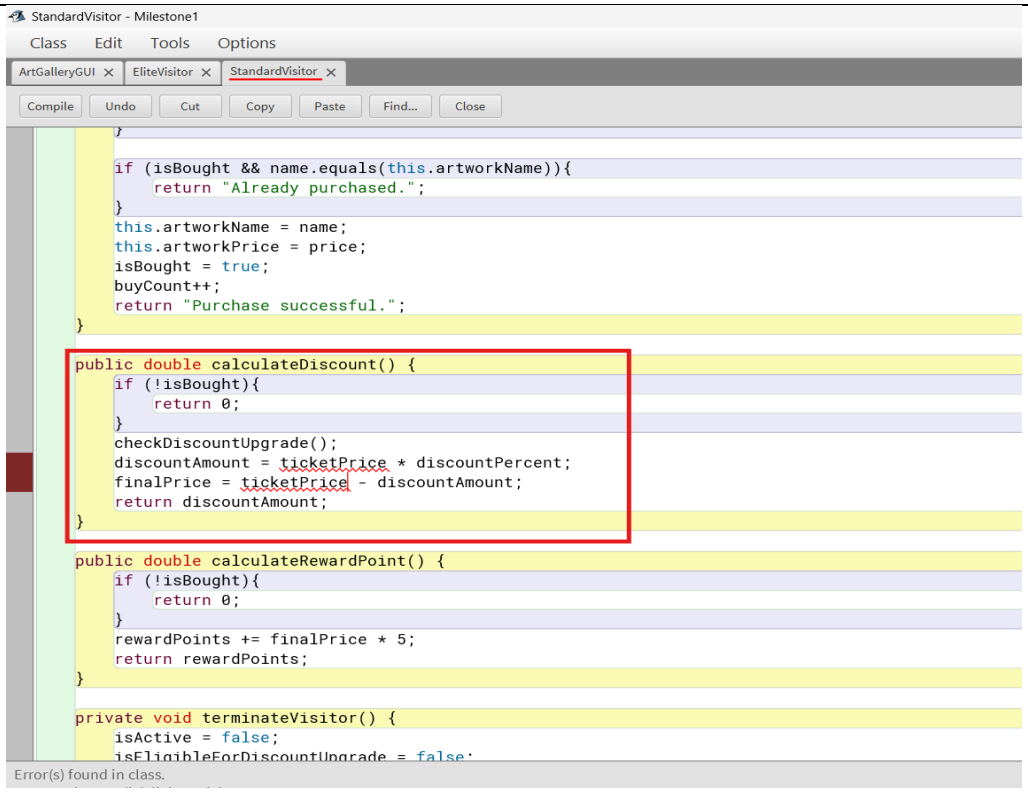
Logic Error	
Description	In StandardVisitor.calculateDiscount(), I accidentally used ticketCost where artworkPrice should be used. Result: wrong discount is being displayed.
Screenshot	
Correction	Changed line to: discountAmount = artworkPrice * discountPercent;

Figure 30: Logical Error

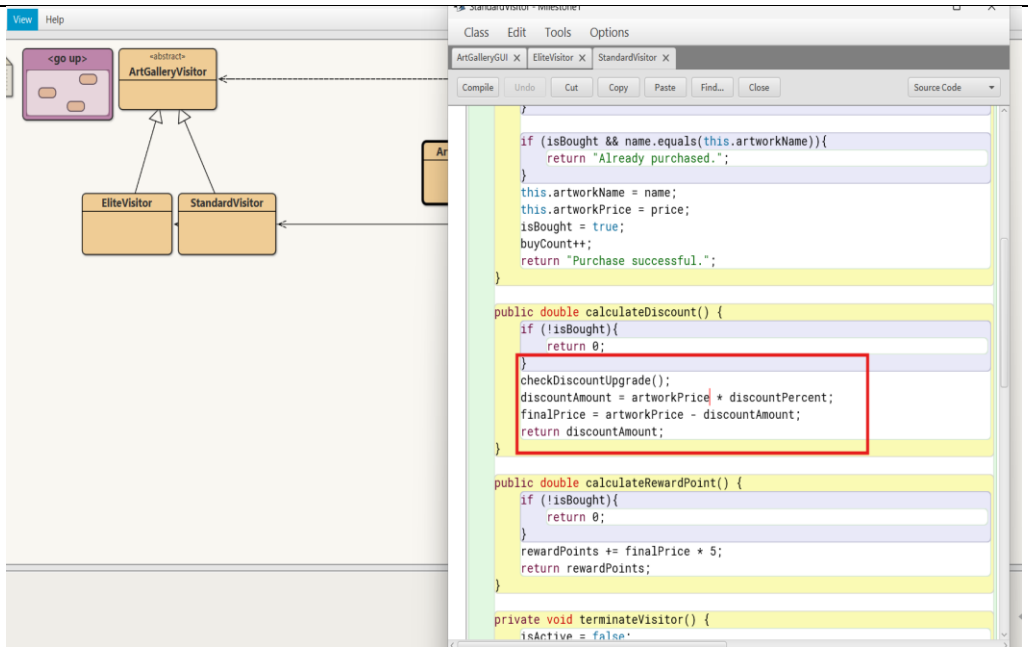
Screenshot	
Reflection	Logically incorrect bugs do not cause compilation failures, so testing + expected vs actual output helped in identifying and resolving them.

Figure 31: Logical Error found

Table 15: Logical error and solved.

Conclusion

Developing the Art Gallery Management System was an insightful process that reinforced my understanding of object-oriented programming with abstraction, inheritance, polymorphism, and encapsulation in the ArtGalleryVisitor hierarchy. The Swing-based Graphical User Interface presented a dynamic, interactive interface for processing visits, purchases, discounts, rewards, billing, and file operations, while exception handling and file persistence ensured system reliability. During the project, certain problems such as handling invalid cancellations, simplicity of the GUI, and avoidance of logical faults in calculations were encountered, but these problems were eliminated through systematic debugging, conditional checks, step-by-step development, and thorough testing. The project was successfully done with its desired objectives and further enhanced my technical skills on OOP, GUI modeling, exception handling, and software testing, which will be of immense use for future software developments.

Appendix

ArtGalleryVisitor.java

```
package ArtGalleryVisitior;


public abstract class ArtGalleryVisitor {

    protected int visitorId, visitCount, cancelCount, buyCount;

    protected String fullName, gender, contactNumber, registrationDate, ticketType, cancellationReason, artworkName;

    protected double ticketCost, rewardPoints, finalPrice, discountAmount, artworkPrice, refundableAmount;

    protected final int cancelLimit = 3;

    protected boolean isActive, isBought;


    //constructor

    public ArtGalleryVisitor(int visitorId, String fullName, String gender, String contactNumber,String registrationDate, double ticketPrice, String ticketType) {

        // visitor information (identity)

        this.visitorId = visitorId;

        this.fullName = fullName;

        this.gender = gender;

        this.contactNumber = contactNumber;

        this.registrationDate = registrationDate;

        this.ticketCost = ticketPrice;

        this.ticketType = ticketType;


        // ticket and financial details

        this.visitCount = 0;

        this.cancelCount = 0;

        this.buyCount = 0;
```

```
        this.rewardPoints = 0;
        this.finalPrice = 0;
        this.discountAmount = 0;
        this.refundableAmount = 0;

        // activity tracking
        this.isActive = false;
        this.isBought = false;
        this.cancellationReason = "";
        this.artworkName = null;
    }

    public void logVisit() {
        visitCount++;  // increase visit count
        isActive = true;  // mark the visitor as active
    }

    //Abstract methods
    public abstract String buyProduct(String name, double price);
    public abstract double calculateDiscount();
    public abstract double calculateRewardPoint();
    public abstract String cancelProduct(String name, String reason);
    public abstract void generateBill();

    //display visitor info
    public void display() {
        System.out.println("ID: " + visitorId + " Name: " + fullName + " Visits: " + visitCount
+" Reward: " + rewardPoints + " Bought: " + isBought);
    }
```

```
    //  Getter methods (needed for GUI)
    public int getVisitorId() {
        return visitorId;
    }
    public String getFullName() {
        return fullName;
    }
    public String getGender() {
        return gender;
    }
    public String getContactNumber() {
        return contactNumber;
    }
    public String getRegistrationDate() {
        return registrationDate;
    }
    public String getTicketType() {
        return ticketType;
    }
    public double getTicketPrice() {
        return ticketCost;
    }
    public String getCancellationReason() {
        return cancellationReason;
    }
    public String getArtworkName() {
        return artworkName;
```

```
    }
    public double getArtworkPrice() {
        return artworkPrice;
    }
    public boolean isActive() {
        return isActive;
    }
    public boolean isBought() {
        return isBought;
    }

    //Setter method
    public void setFullName(String fullName) {
        this.fullName = fullName;
    }

    public void setGender(String gender) {
        this.gender = gender;
    }

    public void setContactNumber(String contactNumber) {
        this.contactNumber = contactNumber;
    }
}
```

StandardVisitor.java

```
package ArtGalleryVisitior;
```

```
public class StandardVisitor extends ArtGalleryVisitor {
    private boolean isEligibleForDiscountUpgrade;
    private final int visitLimit = 5;
    private float discountPercent;

    //constructor
    // Calls parent constructor and sets default discount 10%
    public StandardVisitor(int visitorId, String fullName, String gender, String
contactNumber, String registrationDate, double ticketPrice, String ticketType) {
        super(visitorId, fullName, gender, contactNumber, registrationDate, ticketPrice,
ticketType);
        this.discountPercent = 0.10f;//start with 10% discount
        this.isEligibleForDiscountUpgrade = false;
    }

    // Check if visitor should get discount upgrade
    // If visit count >= 5, discount changes from 10% , 15%
    public boolean checkDiscountUpgrade() {
        if (visitCount >= visitLimit) {
            isEligibleForDiscountUpgrade = true;
            discountPercent = 0.15f;//upgrade to 15%
        }
        return isEligibleForDiscountUpgrade;
    }

    //method for buying an artwork
    public String buyProduct(String name, double price) {
        if (!isActive){
```

```
            return "Log visit first.";
        }

        //prevent duplicate purchase of same artwork
        if (isBought && name.equals(this.artworkName)){
            return "Already purchased.";
        }
        this.artworkName = name;
        this.artworkPrice = price;
        isBought = true;
        buyCount++;
        return "Purchase successful.";
    }

    //Calculate discount for purchased artwork
    public double calculateDiscount() {
        if (!isBought){
            return 0;
        }
        checkDiscountUpgrade();// check if discount should be upgraded
        discountAmount = artworkPrice * discountPercent;
        finalPrice = artworkPrice - discountAmount;
        return discountAmount;
    }

    public double calculateRewardPoint() {
        if (!isBought){
            return 0;// no reward if no purchase
```

```
        }
        rewardPoints += finalPrice * 5;
        return rewardPoints;
    }

     // Terminate visitor account
    // Reset all data if cancellation limit exceeded
    private void terminateVisitor() {
        isActive = false;
        isEligibleForDiscountUpgrade = false;
        visitCount = 0;
        cancelCount = 0;
        rewardPoints = 0;
    }

    public String cancelProduct(String name, String reason) {
        if (cancelCount >= cancelLimit) {
            terminateVisitor();
            return "Account terminated.";
        }

        if (!isBought || !name.equals(this.artworkName)){
            return "No product/cannot cancel.";
        }

        isBought = false;
        this.artworkName = null;
        this.cancellationReason = reason;
```

```
        refundableAmount = artworkPrice - artworkPrice * 0.10;
        rewardPoints -= finalPrice * 5;
        cancelCount++;
        buyCount--;

        return "Cancelled. Refund: " + refundableAmount;
    }

    public void generateBill() {
        if (!isBought) {
            System.out.println("No purchase.");
            return;
        }
        System.out.println("=========== BILL ===========");
        System.out.printf("Visitor ID     : %d%n", visitorId);
        System.out.printf("Visitor Name   : %s%n", fullName);
        System.out.printf("Artwork Name   : %s%n", artworkName);
        System.out.printf("Artwork Price  : %.2f%n", artworkPrice);
        System.out.printf("Discount Amount: %.2f%n", discountAmount);
        System.out.printf("Final Price    : %.2f%n", finalPrice);
        System.out.println("============================");
    }

    public void display() {
        super.display();//dispaly common data from parent class
        System.out.println("DiscountUpgrade: " + isEligibleForDiscountUpgrade + " %: " +
discountPercent);
    }
    // getter for GUI
```

```
    public boolean isDiscountUpgraded() {
        return isEligibleForDiscountUpgrade;
    }
}
```

EliteVisitor.java

```
package ArtGalleryVisitior;


//EliteVisitor class inherits from ArtGalleryVisitor
public class EliteVisitor extends ArtGalleryVisitor {
    private boolean assignedPersonalArtAdvisor;
    private boolean exclusiveEventAccess;


    //constructor
    public EliteVisitor(int visitorId, String fullName, String gender, String contactNumber, String registrationDate, double ticketPrice, String ticketType) {
        super(visitorId, fullName, gender, contactNumber, registrationDate, ticketPrice, ticketType);
        assignedPersonalArtAdvisor = false;
        exclusiveEventAccess = false;
    }


    //Method to assign personal art advisor if visitor has more than 5000 reward points
    public boolean assignPersonalArtAdvisor() {
        if (rewardPoints > 5000){
            assignedPersonalArtAdvisor = true;
        }
        return assignedPersonalArtAdvisor;
    }
```

```
// Method to give exclusive event access if advisor is already assigned
public boolean exclusiveEventAccess() {
    if (assignedPersonalArtAdvisor){
        exclusiveEventAccess = true;
    }
    return exclusiveEventAccess;
}

 // Method for buying artwork
public String buyProduct(String name, double price) {
    if (!isActive){
        return "Log visit first.";
    }
    if (isBought && name.equals(this.artworkName)){
        return "Already purchased.";
    }
    this.artworkName = name;// store artwork name
    this.artworkPrice = price; // store artwork price
    isBought = true;  // mark product as bought
    buyCount++;// increase purchase count

    return "Purchase successful.";
}

public double calculateDiscount() {
    if (!isBought){
        return 0;
```

```
        }
        discountAmount = artworkPrice * 0.40;
        finalPrice = artworkPrice - discountAmount;
        return discountAmount;
    }

    public double calculateRewardPoint() {
        if (!isBought){
            return 0;
        }
        rewardPoints += finalPrice * 10;
        return rewardPoints;
    }

    private void terminateVisitor() {
        isActive = false;
        assignedPersonalArtAdvisor = false;
        exclusiveEventAccess = false;
        visitCount = 0;
        cancelCount = 0;
        rewardPoints = 0;
    }

    public String cancelProduct(String name, String reason) {
        if (cancelCount >= cancelLimit) {
            terminateVisitor();
            return "Account terminated."; //too many cancellation
        }
```

```
        if (!isBought || !name.equals(this.artworkName)){
            return "No product/cannot cancel.";
        }

        isBought = false;
        this.artworkName = null;
        this.cancellationReason = reason;

        refundableAmount = artworkPrice - artworkPrice * 0.05;
        rewardPoints -= finalPrice * 10;
        cancelCount++;
        buyCount--;

        return "Cancelled. Refund: " + refundableAmount;
    }

    public void generateBill() {
        if (!isBought) {
            System.out.println("No purchase.");
            return;
        }
        System.out.println("=========== BILL ===========");
        System.out.printf("Visitor ID     : %d%n", visitorId);
        System.out.printf("Visitor Name   : %s%n", fullName);
        System.out.printf("Artwork Name   : %s%n", artworkName);
        System.out.printf("Artwork Price  : %.2f%n", artworkPrice);
        System.out.printf("Discount Amount: %.2f%n", discountAmount);
        System.out.printf("Final Price    : %.2f%n", finalPrice);
```

```
        System.out.println("============================");
    }

    public void display() {
        super.display(); // call parent class display method
        System.out.println("Art Advisor: " + assignedPersonalArtAdvisor + " Exclusive
Event: " + exclusiveEventAccess);
    }
    // getters for GUI
    public boolean hasAdvisor() {
        return assignedPersonalArtAdvisor;
    }
    public boolean hasExclusiveAccess() {
        return exclusiveEventAccess;
    }
}
```

ArtGalleryGUI.java

```
package ArtGalleryVisitior;


import javax.swing.*;
import java.awt.*;
import java.awt.event.*;
import java.io.*;
import java.util.*;
import javax.swing.border.TitledBorder;
import javax.swing.plaf.FontUIResource;
import javax.swing.table.DefaultTableModel;
```

```
// Custom Exception for Duplicate Visitor ID
class DuplicateVisitorIdException extends Exception {
    public DuplicateVisitorIdException(String msg) {
        super(msg);
    }
}

public class ArtGalleryGUI extends JFrame implements ActionListener {
    private JTextField idText, fullNameText, contactNumberText, ticketPriceText,
artworkNameText, expensesValueText, withdrawalReasonsText;
    private JRadioButton maleBtn, femaleBtn, otherBtn;
    private ButtonGroup genderGroup;
    private JComboBox<String> dayCombo, monthCombo, yearCombo, ticketTypeBox;
    private JButton addVisitorBtn, logVisitBtn, buyProductBtn, assignAdvisorBtn,
checkUpgradeBtn,calcDiscountBtn, calcRewardBtn, cancelProductBtn, billBtn,
displayBtn,clearBtn, saveFileBtn, readFileBtn, exitBtn, eventAccessBtn;
    private final ArrayList<ArtGalleryVisitor> visitorList = new ArrayList<>();

    // Global font
    private static void setUIFont(FontUIResource f, Color textColor) {
        Enumeration<Object> keys = UIManager.getDefaults().keys();
        while (keys.hasMoreElements()) {
            Object key = keys.nextElement();
            Object value = UIManager.get(key);
            if (value instanceof FontUIResource) UIManager.put(key, f);
            if (key.toString().toLowerCase().contains("foreground")) UIManager.put(key,
textColor);
        }
```

```
}

public ArtGalleryGUI() {
    //// set font + window properties
    setUIFont(new FontUIResource(new Font("Arial", Font.PLAIN, 14)), Color.BLACK);
    setTitle("Art Gallery Management System");
    setSize(800, 800);
    setDefaultCloseOperation(EXIT_ON_CLOSE);
    setLocationRelativeTo(null);

    // Common sizes
    Dimension btnSize = new Dimension(180, 32);
    Dimension tfSize  = new Dimension(180, 24);

    // MAIN CONTAINER (6 stacked panels)
    JPanel mainPanel = new JPanel();
    mainPanel.setLayout(new BoxLayout(mainPanel, BoxLayout.Y_AXIS));
    mainPanel.setBorder(BorderFactory.createEmptyBorder(12, 15, 12, 15));
    mainPanel.setBackground(Color.WHITE);

    // ROW 1: Visitor Information
    JPanel visitorPanel = createSectionPanel("Visitor Information");
    visitorPanel.setLayout(new GridBagLayout());
    GridBagConstraints g = gbc(6, 6);
    int rowL = 0, rowR = 0;

    addLabel(visitorPanel, "Visitor ID:", g, 0, rowL);
    idText = new JTextField();
```

```
        idText.setPreferredSize(tfSize);
        // only allow numbers
        idText.addKeyListener(new KeyAdapter() {
            public void keyTyped(KeyEvent e) {
                if (!Character.isDigit(e.getKeyChar()) && e.getKeyChar() != '\b') {
                    e.consume();
                    JOptionPane.showMessageDialog(null, "Only numbers allowed in Visitor
ID!");
                }
            }
        });
        addField(visitorPanel, idText, g, 1, rowL++);

        addLabel(visitorPanel, "Full Name:", g, 0, rowL);
        fullNameText = new JTextField();
        fullNameText.setPreferredSize(tfSize);
        // only allow letters
        fullNameText.addKeyListener(new KeyAdapter() {
            public void keyTyped(KeyEvent e) {
                if (!Character.isLetter(e.getKeyChar()) && e.getKeyChar() != ' ' &&
e.getKeyChar() != '\b') {
                    e.consume();
                    JOptionPane.showMessageDialog(null, "Only letters allowed in Full
Name!");
                }
            }
        });
        // Gender radio buttons
        addField(visitorPanel, fullNameText, g, 1, rowL++);
```

```
        addLabel(visitorPanel, "Gender:", g, 0, rowL);
        JPanel genderPanel = new JPanel(new FlowLayout(FlowLayout.LEFT, 6, 0));
        genderPanel.setOpaque(false);
        maleBtn = new JRadioButton("Male");
        femaleBtn = new JRadioButton("Female");
        otherBtn = new JRadioButton("Others");
        genderGroup = new ButtonGroup();
        genderGroup.add(maleBtn); genderGroup.add(femaleBtn);
genderGroup.add(otherBtn);
        genderPanel.add(maleBtn); genderPanel.add(femaleBtn);
genderPanel.add(otherBtn);
        addField(visitorPanel, genderPanel, g, 1, rowL++);


        addLabel(visitorPanel, "Contact Number:", g, 0, rowL);
        contactNumberText = new JTextField();
        contactNumberText.setPreferredSize(tfSize);
         // only digits allowed
        contactNumberText.addKeyListener(new KeyAdapter() {
            public void keyTyped(KeyEvent e) {
                if (!Character.isDigit(e.getKeyChar()) && e.getKeyChar() != '\b') {
                    e.consume();
                    JOptionPane.showMessageDialog(null, "Only numbers allowed in Contact
Number!");
                }
            }
        });
        addField(visitorPanel, contactNumberText, g, 1, rowL++);
        addLabel(visitorPanel, "Registration Date:", g, 0, rowL);
```

```
        String[] days = new String[31]; for (int i = 1; i <= 31; i++) days[i - 1] =
String.valueOf(i);
        dayCombo = new JComboBox<>(days);
        String[] months =
{"Jan","Feb","Mar","Apr","May","Jun","Jul","Aug","Sep","Oct","Nov","Dec"};
        monthCombo = new JComboBox<>(months);
        String[] years = {"2022","2023","2024","2025"};
        yearCombo = new JComboBox<>(years);
        JPanel datePanel = new JPanel(new FlowLayout(FlowLayout.LEFT, 6, 0));
        datePanel.setOpaque(false);
        datePanel.add(dayCombo); datePanel.add(monthCombo);
datePanel.add(yearCombo);
        addField(visitorPanel, datePanel, g, 1, rowL++);


        addLabel(visitorPanel, "Ticket Type:", g, 2, rowR);
        ticketTypeBox = new JComboBox<>(new String[]{"Standard", "Elite"});
        ((JComponent)ticketTypeBox).setPreferredSize(tfSize);
        addField(visitorPanel, ticketTypeBox, g, 3, rowR++);


        addLabel(visitorPanel, "Artwork Name:", g, 2, rowR);
        artworkNameText = new JTextField();
        artworkNameText.setPreferredSize(tfSize);
        addField(visitorPanel, artworkNameText, g, 3, rowR++);


        addLabel(visitorPanel, "Artwork Price:", g, 2, rowR);
        expensesValueText = new JTextField();
        expensesValueText.setPreferredSize(tfSize);
        addField(visitorPanel, expensesValueText, g, 3, rowR++);
```

```
        addLabel(visitorPanel, "Ticket Price:", g, 2, rowR);
        ticketPriceText = new JTextField();
        ticketPriceText.setPreferredSize(tfSize);
        addField(visitorPanel, ticketPriceText, g, 3, rowR++);

        addLabel(visitorPanel, "Cancellation Reason:", g, 2, rowR);
        withdrawalReasonsText = new JTextField();
        withdrawalReasonsText.setPreferredSize(tfSize);
        addField(visitorPanel, withdrawalReasonsText, g, 3, rowR++);

        addVisitorBtn = makeButton("Add Visitor", btnSize);
        addVisitorBtn.addActionListener(this);
        GridBagConstraints gBtn = gbc(6, 6);
        gBtn.gridx = 2; gBtn.gridy = Math.max(rowL, rowR) + 1;
        gBtn.gridwidth = 2;
        gBtn.anchor = GridBagConstraints.SOUTHEAST;
        visitorPanel.add(addVisitorBtn, gBtn);

        ticketTypeBox.addActionListener(e -
>ticketPriceText.setText(Objects.equals(ticketTypeBox.getSelectedItem(), "Standard") ?
"1000" : "2000"));

        // ROW 2: Visit Management
        JPanel visitPanel = createButtonRowPanel("Visit Management", 2);
        logVisitBtn = makeButton("Log Visit", btnSize);
        logVisitBtn.addActionListener(this);
        displayBtn = makeButton("Display Visitor Details", btnSize);
        displayBtn.addActionListener(this);
        addButtonsHorizontally(visitPanel, logVisitBtn, displayBtn);
```

```
        // ROW 3: Purchase Management
        JPanel purchasePanel = createButtonRowPanel("Purchase Management", 3);
        buyProductBtn = makeButton("Buy Product", btnSize);
        buyProductBtn.addActionListener(this);
        cancelProductBtn = makeButton("Cancel Product", btnSize);
        cancelProductBtn.addActionListener(this);
        billBtn = makeButton("Generate Bill", btnSize);
        billBtn.addActionListener(this);
        addButtonsHorizontally(purchasePanel, buyProductBtn, cancelProductBtn, billBtn);

        // ROW 4: Benefits & Rewards (FIXED to match sizes)
        JPanel benefitsPanel = createButtonRowPanel("Benefits & Rewards", 4);
        benefitsPanel.setLayout(new GridLayout(2, 2, 10, 10));
        calcDiscountBtn = makeButton("Calculate Discount", btnSize);
        calcDiscountBtn.addActionListener(this);

        calcRewardBtn = makeButton("Reward Points", btnSize);
        calcRewardBtn.addActionListener(this);

        checkUpgradeBtn = makeButton("Check Upgrade", btnSize);
        checkUpgradeBtn.addActionListener(this);

        assignAdvisorBtn = makeButton("Personal Art Advisor", btnSize);
        assignAdvisorBtn.addActionListener(this);

        eventAccessBtn = makeButton("Exclusive Event Access", new Dimension(180,
32));
        eventAccessBtn.addActionListener(this);
```

```
        addButtonsHorizontally(benefitsPanel, calcDiscountBtn, calcRewardBtn,
checkUpgradeBtn, assignAdvisorBtn, eventAccessBtn);

        // ROW 5: File Operations
        JPanel filePanel = createButtonRowPanel("File Operations", 5);
        saveFileBtn = makeButton("Save File", btnSize);
        saveFileBtn.addActionListener(this);
        readFileBtn = makeButton("Read File", btnSize);
        readFileBtn.addActionListener(this);
        addButtonsHorizontally(filePanel, saveFileBtn, readFileBtn);

        // ROW 6: Others
        JPanel othersPanel = createButtonRowPanel("Others", 6);
        clearBtn = makeButton("Clear Fields", btnSize);
        clearBtn.addActionListener(this);
        exitBtn = makeButton("Exit", btnSize);
        exitBtn.addActionListener(this);
        addButtonsHorizontally(othersPanel, clearBtn, exitBtn);

        mainPanel.add(visitorPanel);
        mainPanel.add(Box.createVerticalStrut(12));
        mainPanel.add(visitPanel);
        mainPanel.add(Box.createVerticalStrut(12));
        mainPanel.add(purchasePanel);
        mainPanel.add(Box.createVerticalStrut(12));
        mainPanel.add(benefitsPanel);
        mainPanel.add(Box.createVerticalStrut(12));
        mainPanel.add(filePanel);
```

```
        mainPanel.add(Box.createVerticalStrut(12));
        mainPanel.add(othersPanel);

        add(mainPanel);
        setVisible(true);
    }

    // UI Helpers
    private JPanel createSectionPanel(String title) {
        JPanel panel = new JPanel();
        panel.setBackground(Color.WHITE);
        panel.setBorder(BorderFactory.createTitledBorder(
                BorderFactory.createLineBorder(new Color(70,70,70), 2),
                title, TitledBorder.LEFT, TitledBorder.TOP,
                new Font("Arial", Font.BOLD, 14), Color.BLACK
        ));
        panel.setAlignmentX(Component.LEFT_ALIGNMENT);
        return panel;
    }

    private JPanel createButtonRowPanel(String title, int rowIndex) {
        JPanel p = createSectionPanel(title);
        p.setLayout(new GridBagLayout());
        return p;
    }

    private JButton makeButton(String text, Dimension size) {
        JButton b = new JButton(text);
```

```
        b.setPreferredSize(size);
        b.setMinimumSize(size);
        b.setMaximumSize(size);
        b.setFocusPainted(false);
        return b;
    }

    private GridBagConstraints gbc(int ix, int iy) {
        GridBagConstraints g = new GridBagConstraints();
        g.insets = new Insets(ix, ix, iy, iy);
        g.anchor = GridBagConstraints.WEST;
        g.fill = GridBagConstraints.HORIZONTAL;
        g.weightx = 1.0;
        return g;
    }

    private void addLabel(JPanel panel, String text, GridBagConstraints g, int x, int y) {
        GridBagConstraints c = (GridBagConstraints) g.clone();
        c.gridx = x; c.gridy = y;
        c.weightx = 0.0;
        JLabel lbl = new JLabel(text);
        panel.add(lbl, c);
    }

    private void addField(JPanel panel, JComponent comp, GridBagConstraints g, int x,
int y) {
        GridBagConstraints c = (GridBagConstraints) g.clone();
        c.gridx = x; c.gridy = y;
        c.weightx = 1.0;
```

```
        panel.add(comp, c);
    }

    private void addButtonsHorizontally(JPanel panel, JButton... buttons) {
        GridBagConstraints c = gbc(6, 6);
        for (int i = 0; i < buttons.length; i++) {
            c.gridx = i;
            c.gridy = 0;
            c.weightx = 0.0;
            panel.add(buttons[i], c);
        }
        c.gridx = buttons.length;
        c.weightx = 1.0;
        panel.add(Box.createHorizontalGlue(), c);
    }

    //  Action handling for buttons
    @Override
    public void actionPerformed(ActionEvent e) {
        try {
            Object src = e.getSource();
            if (src == addVisitorBtn) {
                addVisitor();
            } else if (src == logVisitBtn) {
                logVisit();
            } else if (src == buyProductBtn) {
                buyProduct();
            } else if (src == assignAdvisorBtn) {
```

```
            assignAdvisor();
        } else if (src == checkUpgradeBtn) {
            checkUpgrade();
        }else if  (src == eventAccessBtn){
            checkEventAccess();
        }else if (src == calcDiscountBtn) {
            calculateDiscount();
        } else if (src == calcRewardBtn) {
            calculateRewardPoints();
        } else if (src == cancelProductBtn) {
            cancelProduct();
        } else if (src == billBtn) {
            generateBillToFile();
        } else if (src == displayBtn) {
            displayVisitorDetails();
        } else if (src == clearBtn) {
            clearFields();
        } else if (src == saveFileBtn) {
            saveToFile();
        } else if (src == readFileBtn) {
            readFromFile();
        } else if (src == exitBtn) {
            System.exit(0);
        }
    } catch (DuplicateVisitorIdException dupe) {
        JOptionPane.showMessageDialog(this, dupe.getMessage(), "Duplicate ID",
JOptionPane.WARNING_MESSAGE);
    } catch (NumberFormatException nfe) {
```

```
            JOptionPane.showMessageDialog(this, "Please enter valid numeric values
where required.", "Invalid Number", JOptionPane.ERROR_MESSAGE);
        } catch (FileNotFoundException fnf) {
            JOptionPane.showMessageDialog(this, "File not found: " + fnf.getMessage(),
"File Error", JOptionPane.ERROR_MESSAGE);
        } catch (IOException ioe) {
            JOptionPane.showMessageDialog(this, "I/O Error: " + ioe.getMessage(), "I/O
Error", JOptionPane.ERROR_MESSAGE);
        } catch (Exception ex) {
            JOptionPane.showMessageDialog(this, "Error: " + ex.getMessage(), "Error",
JOptionPane.ERROR_MESSAGE);
        }
    }

    //Helper: find visitor by ID (throws if not found) ---
    private ArtGalleryVisitor findVisitorById() throws Exception {
        String idStr = idText.getText().trim();
        if (idStr.isEmpty()) throw new Exception("Enter Visitor ID first.");
        int id = Integer.parseInt(idStr);
        for (ArtGalleryVisitor v : visitorList) {
            if (v.getVisitorId() == id) return v;
        }
        throw new Exception("Visitor ID " + id + " not found.");
    }

    // 1) Add Visitor (with duplicate check)
    private void addVisitor() throws Exception {
        if (idText.getText().trim().isEmpty() || fullNameText.getText().trim().isEmpty()
                || contactNumberText.getText().trim().isEmpty() ||
ticketPriceText.getText().trim().isEmpty()) {
```

```
            throw new Exception("Please fill all required fields (ID, Name, Contact, Ticket
Price).");
        }


        int id = Integer.parseInt(idText.getText().trim());


        // duplicate ID check
        for (ArtGalleryVisitor v : visitorList) {
            if (v.getVisitorId() == id) throw new DuplicateVisitorIdException("Visitor ID
already exists!");
        }


        String name = fullNameText.getText().trim();
        String gender = maleBtn.isSelected() ? "Male" : (femaleBtn.isSelected() ? "Female"
: "Others");
        String number = contactNumberText.getText().trim();
        String date = dayCombo.getSelectedItem() + "/" + monthCombo.getSelectedItem()
+ "/" + yearCombo.getSelectedItem();
        String ticketType = (String) ticketTypeBox.getSelectedItem();
        double ticketPrice = Double.parseDouble(ticketPriceText.getText().trim());


        ArtGalleryVisitor visitor = ticketType.equals("Standard")
                ? new StandardVisitor(id, name, gender, number, date, ticketPrice,
ticketType)
                : new EliteVisitor(id, name, gender, number, date, ticketPrice, ticketType);


        visitorList.add(visitor);
        JOptionPane.showMessageDialog(this, "Visitor Added Successfully!");
    }
```

```
    // 2) Log Visit
    private void logVisit() throws Exception {
        ArtGalleryVisitor v = findVisitorById();
        v.logVisit();
        JOptionPane.showMessageDialog(this, "Visit logged. Total visits: " + v.visitCount);
    }

    // 3) Buy Product
    private void buyProduct() throws Exception {
        ArtGalleryVisitor v = findVisitorById();
        String name = artworkNameText.getText().trim();
        if (name.isEmpty()) throw new Exception("Enter Artwork Name.");
        if (expensesValueText.getText().trim().isEmpty()) throw new Exception("Enter
Artwork Price.");
        double price = Double.parseDouble(expensesValueText.getText().trim());

        String msg = v.buyProduct(name, price);
        JOptionPane.showMessageDialog(this, msg);
    }

    // 4) Assign Personal Art Advisor (Elite only)
    private void assignAdvisor() throws Exception {
        ArtGalleryVisitor v = findVisitorById();
        if (v instanceof EliteVisitor) {
            EliteVisitor ev = (EliteVisitor) v;
            boolean assigned = ev.assignPersonalArtAdvisor();
            JOptionPane.showMessageDialog(this,
                    assigned ? "Personal Art Advisor assigned." : "Not eligible yet (need >
5000 reward points).");
```

```
        } else {
            JOptionPane.showMessageDialog(this, "Only Elite visitors can be assigned a
Personal Art Advisor.");
        }
    }

    // 5) Check Upgrade (Standard only)
    private void checkUpgrade() throws Exception {
        ArtGalleryVisitor v = findVisitorById();
        if (v instanceof StandardVisitor) {
            StandardVisitor sv = (StandardVisitor) v;
            boolean upgraded = sv.checkDiscountUpgrade();
            JOptionPane.showMessageDialog(this,
                    upgraded ? "Upgrade achieved! Discount increased." : "Not upgraded yet.
Visit more times.");
        } else {
            JOptionPane.showMessageDialog(this, "Only Standard visitors have discount
upgrade checks.");
        }
    }

    // 6) Event Access
    private void checkEventAccess() throws Exception {
        ArtGalleryVisitor v = findVisitorById();
        if (v instanceof EliteVisitor) {
            EliteVisitor ev = (EliteVisitor) v;
            boolean access = ev.exclusiveEventAccess();
            JOptionPane.showMessageDialog(this,
                access ? "This Elite visitor has Exclusive Event Access!" : "No exclusive
access yet.");
```

```
        } else {
            JOptionPane.showMessageDialog(this, "Only Elite visitors can have Event Access.");
        }
    }

    // 7) Calculate Discount
    private void calculateDiscount() throws Exception {
        ArtGalleryVisitor v = findVisitorById();
        double d = v.calculateDiscount();
        JOptionPane.showMessageDialog(this, "Discount calculated: " + String.format("%.2f", d));
    }

    // 8) Calculate Reward Points
    private void calculateRewardPoints() throws Exception {
        ArtGalleryVisitor v = findVisitorById();
        double rp = v.calculateRewardPoint();
        JOptionPane.showMessageDialog(this, "Reward Points: " + String.format("%.2f", rp));
    }

    // 9) Cancel Product
    private void cancelProduct() throws Exception {
        ArtGalleryVisitor v = findVisitorById();
        String name = artworkNameText.getText().trim();
        String reason = withdrawalReasonsText.getText().trim();
        if (name.isEmpty()) throw new Exception("Enter Artwork Name to cancel.");
        if (reason.isEmpty()) throw new Exception("Enter Cancellation Reason.");
```

```
        String msg = v.cancelProduct(name, reason);
        JOptionPane.showMessageDialog(this, msg);
    }

    // 10) Generate Bill (call visitor.generateBill() and also export to .txt)
    private void generateBillToFile() throws Exception {
        ArtGalleryVisitor v = findVisitorById();
        // Ensure discount/reward are updated if needed
        v.calculateDiscount();
        v.calculateRewardPoint();

        // Call provided method (prints to console)
        v.generateBill();

        // Export to txt with file handling & try-catch-finally
        String fileName = "Bill_Visitor_" + v.getVisitorId() + ".txt";
        PrintWriter pw = null;
        try {
            pw = new PrintWriter(new BufferedWriter(new FileWriter(fileName)));
            pw.println("=========== ART GALLERY BILL ===========");
            pw.println("Visitor ID         : " + v.getVisitorId());
            pw.println("Name               : " + v.getFullName());
            pw.println("Ticket Type        : " + v.getTicketType());
            pw.println("Ticket Price       : " + String.format("%.2f", v.getTicketPrice()));
            pw.println("---------------------------------------");
            pw.println("Artwork Name       : " + (v.getArtworkName() == null ? "-" :
v.getArtworkName()));
            pw.println("Artwork Price      : " + String.format("%.2f", v.getArtworkPrice()));
            // Access protected fields inside same package:
```

```
            pw.println("Discount Amount    : " + String.format("%.2f", v.discountAmount));
            pw.println("Final Price        : " + String.format("%.2f", v.finalPrice));
            pw.println("Reward Points      : " + String.format("%.2f", v.rewardPoints));
            pw.println("---------------------------------------");
            pw.println("Active Status      : " + v.isActive());
            pw.println("Bought Status      : " + v.isBought());
            pw.println("Cancellations      : " + v.cancelCount);
            pw.println("=======================================");
            JOptionPane.showMessageDialog(this, "Bill exported as: " + fileName);
        } catch (FileNotFoundException fnf) {
            throw fnf; // rethrow to show in UI
        } catch (IOException ioe) {
            throw ioe;
        } finally {
            if (pw != null) pw.close();
        }

        // Also display bill content in a GUI dialog
        showTextFileInDialog(fileName, "Generated Bill");
    }

    // 11) Display Visitor Details (nice table or single visitor info)
    private void displayVisitorDetails() {
        if (visitorList.isEmpty()) {
            JOptionPane.showMessageDialog(this, "No visitors to display.");
            return;
        }
        String[] columns = {"ID", "Name", "Gender", "Contact", "Reg. Date", "Ticket",
"Price", "Visits", "Buys", "Cancels"};
```

```
        DefaultTableModel model = new DefaultTableModel(columns, 0);
        for (ArtGalleryVisitor v : visitorList) {
            model.addRow(new Object[]{
                    v.getVisitorId(), v.getFullName(), v.getGender(), v.getContactNumber(),
                    v.getRegistrationDate(), v.getTicketType(), v.getTicketPrice(),
                    v.visitCount, v.buyCount, v.cancelCount
            });
        }
        JTable table = new JTable(model);
        table.setAutoCreateRowSorter(true);
        JScrollPane scrollPane = new JScrollPane(table);
        JDialog dialog = new JDialog(this, "Visitor Details", true);
        dialog.add(scrollPane);
        dialog.setSize(820, 420);
        dialog.setLocationRelativeTo(this);
        dialog.setVisible(true);
    }

    // 12) Clear Fields
    private void clearFields() {
        idText.setText("");
        fullNameText.setText("");
        contactNumberText.setText("");
        ticketPriceText.setText("");
        artworkNameText.setText("");
        expensesValueText.setText("");
        withdrawalReasonsText.setText("");
        genderGroup.clearSelection();
```

```
        dayCombo.setSelectedIndex(0);
        monthCombo.setSelectedIndex(0);
        yearCombo.setSelectedIndex(0);
        ticketTypeBox.setSelectedIndex(0);
    }

    // 13) Save to File (formatted table of all visitors)
    private void saveToFile() throws IOException {
        String file = "VisitorDetails.txt";
        PrintWriter writer = null;
        try {
            writer = new PrintWriter(new BufferedWriter(new FileWriter(file)));
            // Header (as per your hint, adjusted to our fields)
            writer.printf("%-5s %-15s %-10s %-15s %-15s %-10s %-12s %-8s %-8s %-
8s%n",
                    "ID", "Name", "Gender", "Phone", "RegDate", "Plan", "TicketCost", "Visits",
"Buys", "Cancels");
            writer.println("----------------------------------------------------------------------------------");
            for (ArtGalleryVisitor v : visitorList) {
                writer.printf("%-5d %-15s %-10s %-15s %-15s %-10s %-12.2f %-8d %-8d %-
8d%n",
                        v.getVisitorId(), v.getFullName(), v.getGender(), v.getContactNumber(),
                        v.getRegistrationDate(), v.getTicketType(), v.getTicketPrice(),
                        v.visitCount, v.buyCount, v.cancelCount);
            }
            JOptionPane.showMessageDialog(this, "Visitor details saved to " + file);
        } finally {
            if (writer != null) writer.close();
        }
```

```
    }

    // 14) Read from File (show previously saved details in GUI)
    private void readFromFile() throws IOException {
        String file = "VisitorDetails.txt";
        // Use BufferedReader (plus show FileNotFound separately)
        File f = new File(file);
        if (!f.exists()) throw new FileNotFoundException(file + " does not exist.");

        StringBuilder sb = new StringBuilder();
        BufferedReader br = null;
        try {
            br = new BufferedReader(new FileReader(f));
            String line;
            while ((line = br.readLine()) != null) sb.append(line).append("\n");
        } finally {
            if (br != null) try { br.close(); } catch (IOException ignored) {}
        }

        JTextArea area = new JTextArea(sb.toString(), 25, 80);
        area.setEditable(false);
        JDialog dlg = new JDialog(this, "VisitorDetails.txt (Read Only)", true);
        dlg.add(new JScrollPane(area));
        dlg.pack();
        dlg.setLocationRelativeTo(this);
        dlg.setVisible(true);
    }
```

```
    // Utility: show a small text file in dialog
    private void showTextFileInDialog(String fileName, String title) {
        try {
            StringBuilder sb = new StringBuilder();
            Scanner sc = new Scanner(new File(fileName));
            while (sc.hasNextLine()) sb.append(sc.nextLine()).append("\n");
            sc.close();
            JTextArea area = new JTextArea(sb.toString(), 22, 60);
            area.setEditable(false);
            JDialog dlg = new JDialog(this, title, true);
            dlg.add(new JScrollPane(area));
            dlg.pack();
            dlg.setLocationRelativeTo(this);
            dlg.setVisible(true);
        } catch (Exception ignored) {
            // Silent; file already confirmed written.
        }
    }

    public static void main(String[] args) {
        new ArtGalleryGUI();
    }
}
```